

DL usmeni

1. [DL01a_Uvod_u_duboko_ucenje] Šta je duboko učenje u kontekstu mašinskog učenja i veštačke inteligencije? Koje su glavne karakteristike klasičnih sistema veštačke inteligencije, mašinskog učenja i dubokog učenja? Grafički ilustrovati i objasniti.

Duboko učenje je specijalni slučaj mašinskog učenja, koji je deo veštačke inteligencije (u nekom širem smislu).



AI - ponašanje determinisano nekim pravilima, deluje pametno, ali ništa tu posebno ne uči
ML - učenje na osnovu primera i podataka, stvar je bila u procesiranju podataka koji su ulaz za algoritam koji uspostavlja veze između ulaza i izlaza. Jako je bitan način predstavljanja podataka (na slici - ekstrakcija fичера)

DL - drugačiji trend (end to end learning), izbacuje se човек iz jednačine, koristi se gotova struktura ili algoritam koja ekstrahuje fичере koji su ulazi za algoritam

Malo lepše:

Duboko učenje (eng. deep learning) je grana mašinskog učenja koja se bavi razvojem algoritama i modela koji oponašaju rad ljudskog mozga, koristeći veštačke neuronske mreže sa više slojeva. Ovaj pristup se naziva "duboko" jer se sastoji od više slojeva veštačkih neurona koji postepeno izvlače složene reprezentacije podataka.

Klasični sistemi veštačke inteligencije se obično oslanjaju na eksplicitno programiranje i ručno definisanje pravila i heuristika za rešavanje problema. Oni se oslanjaju na različite tehnike, poput pretraživanja, logike ili pravila donošenja odluka. Ovi sistemi mogu biti efikasni u rešavanju određenih problema, ali zahtevaju da programeri unapred definišu pravila i predviđaju sve moguće situacije.

Mašinsko učenje predstavlja pristup u kojem algoritmi uče iz podataka i primenjuju stečeno znanje **kako bi donosili odluke ili davali predikcije**. Ovi algoritmi koriste različite tehnike učenja, kao što su nadgledano, nenadgledano i pojačano učenje. Oni izvlače zaključke iz podataka, pronalaze obrasce i donose odluke na osnovu tih obrazaca.

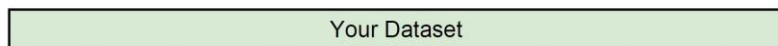
Duboko učenje je poseban oblik mašinskog učenja koji koristi duboke neuronske mreže sa više slojeva. Ove neuronske mreže mogu naučiti reprezentacije podataka sa različitim nivoima apstrakcije, omogućavajući im da samostalno **izvlače složene karakteristike i obrasce iz sirovih podataka**. Ovo je postignuto zahvaljujući unutrašnjem predstavljanju podataka u različitim slojevima mreže, gde svaki sloj obrađuje i izdvaja određene karakteristike.

2. [DL01b_Image_Classification_Pipeline] Šta su hiperparametri i koja metodologija podele podataka na podskupove se preporučuje za nalaženje optimalnih vrednosti?

Hiperparametri - parametri koji se koriste za podešavanje modela dubokog učenja, ali se njihove vrednosti ne uče direktno iz podataka za obučavanje, već se zadaju algoritmu, zavise od konkretnog problema, ideja je isprobati više kombinacija kako bi se video šta daje najbolje rezultate. Oni kontrolišu ponašanje samog modela, kao što su broj slojeva u neuronskoj mreži, broj neuronskih jedinica u svakom sloju, brzina učenja itd.

Idea #1: Choose hyperparameters that work best on the data

BAD: K = 1 always works perfectly on training data



Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data



Idea #3: Split data into **train**, **val**, and **test**; choose hyperparameters on val and evaluate on test

Better!



Podela na test, trening, validacioni skup:

Trening skup: Ovo je skup podataka koji se koristi za treniranje modela. Model se prilagođava trening podacima kako bi naučio odgovarajuće reprezentacije i pravila. Ovaj skup treba da bude dovoljno reprezentativan za sve klase ili kategorije koje model treba da nauči.

Validacioni skup: Nakon svake epohe ili iteracije tokom treninga, koristi se validacioni skup da bi se procenio performanse modela i izvršila optimizacija hiperparametara. Ovaj skup pomaže u sprečavanju prenaučivosti (eng. overfitting) i omogućava prilagođavanje modela.

Test skup: Kada je model potpuno treniran i optimizovan, koristi se test skup za konačno procenjivanje performansi. Ovaj skup sadrži podatke koje model nije video tokom treninga ili validacije, pružajući objektivnu procenu sposobnosti modela da se generalizuje na nove, nepoznate podatke.

Hiperparametri se daju tako da biraju najbolje parametre na validacionom skupu, a konačni rezultati se odrede na test skupu.

Idea #4: Cross-Validation: Split data into **folds**, try each fold as validation and average the results

fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test

Kad nije previše zahtevno obučavanje, umesto fiksnog validacionog skupa radi se podela na 5 skupova. Ovo je korisno za male skupove podataka, ali ne koristi se previše često u DL. (Kao odgovor na pitanje šta je optimalno - ili podela na skupove za treniranje, testiranje i validaciju, ili KFold, zavisno od skupa)

Malo lepše:

Podela na train i test

Podela na train i test omogućava osnovnu procenu modela, ali može biti podložna varijabilnosti u zavisnosti od podele podataka. Ne znamo kako će se model ponašati nad novim podacima.

Podela na train, validate i test

Podela na train, validate i test omogućava finije podešavanje modela i bolju procenu performansi. Validacioni skup pomaže u izboru najboljih hiperparametara i smanjuje rizik od prenaučivosti.

k-fold Cross-Validation

k-fold Cross-Validation pruža najpouzdaniju procenu modela koristeći sve dostupne podatke na efikasan način. Ova metoda smanjuje varijabilnost procena performansi i omogućava stabilniji i pouzdaniji izbor modela. **Objašnjenje:** U svakoj iteraciji, jedan podskup se koristi kao validacioni

skup, dok se preostalih k-1 podskupova koristi za treniranje. Proces se ponavlja k puta, tako da svaki podskup bude korišćen kao validacioni skup tačno jednom.

3. [DL01b_Image_Classification_Pipeline] Koju matematičku operaciju vrši linearni klasifikator? Kojih dimenzija su parametri koji se koristi za klasifikaciju slika dimenzija 32x32x3 u 10 klasa.

Linearni klasifikatori su gradivni elementi neuronskih mreža. Parametarski pristup, gde je ideja napraviti fju koja zavisi od parametara (W), na ulazu ima sliku, na izlazu skor za svaku od 10 klasa koju određujemo.

Linearni klasifikator primenjuje linearnu funkciju da bi klasifikovao podatke. Ta linearna funkcija može biti predstavljena na sledeći način: $f(x, W) = Wx + b$, gde je:

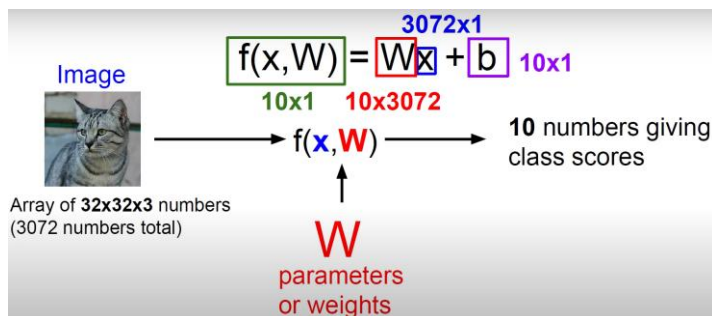
x - vektor ulaznih podataka,

W - vektor težina,

b - pristrasnost (bias)

Wx - skalarni proizvod vektora težina i ulaznih podataka

Izlaz funkcije koristi se za donošenje odluke o klasifikaciji: Najveći skor znači najveća verovatnoća pripadnosti nekoj klasi.



Ako podaci nisu linerano separabilni, ovaj klasifikator neće raditi mnogo dobro. Zato se primenjuje nad fičerima iz slike koji nekako drugačije predstavljaju originalne podatke.

Parametri koji se koriste za klasifikaciju slika dimenzija 32.x32x3 u 10 klasa su matrica težina dimenzija 3072x10 i vektor smestaja dimenzija 10x1.

4. [DL01c_Loss_Functions_and_Optimization] Koja je uloga regularizacionog člana kod funkcije greške? Kako se računa za slučaj L1 i L2 regularizacije.

On je reko: Regularizacija - služi za izbegavanje da težine koje obučavamo (W) dobiju neželjene vrednosti, tj. vrednosti koje su isuviše velike. Model ne treba da radi dobro samo sa trening podacima, već i sa podacima koje ranije nije video.

Regularization

λ = regularization strength
(hyperparameter)

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too well* on training data

Simple examples

L2 regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$

Na slajdu stoje teze:

- **Izražavanje preferenci prema težinama:**
Regularizacija omogućava da se favorizuju određene vrednosti težina, obično manje vrednosti, kako bi se smanjilo prekomerno prilagođavanje modela trening podacima.
- **Učiniti model jednostavnim da bi dobro radio na test podacima:**
Regularizacija pojednostavljuje model, što pomaže u boljoj generalizaciji i performansama na neviđenim test podacima.
- **Poboljšanje optimizacije dodavanjem zakrivljenosti:**
Regularizacija može dodati zakrivljenost funkciji gubitka, čime se poboljšava stabilnost i brzina konvergencije algoritama za optimizaciju, kao što je gradijentni spust.

Malo lepše:

Uloga regularizacionog člana u funkciji greške je da kontroliše kompleksnost modela i spreči previše adaptiranje na trening skupu podataka (overfitting). On dodeljuje kaznu ili ograničava parametre modela, što pomaže u postizanju bolje opšteg predviđanja na novim, neviđenim podacima.

Za slučaj L1 (Laplasova) regularizacije, regularizacioni član se računa kao zbir apsolutnih vrednosti svih parametara modela, pomnožen sa hiperparametrom lambda (λ), koji kontroliše jačinu regularizacije.

Za slučaj L2 (Tihonova) regularizacije, regularizacioni član se računa kao zbir kvadrata svih parametara modela, pomnožen sa hiperparametrom lambda (λ).

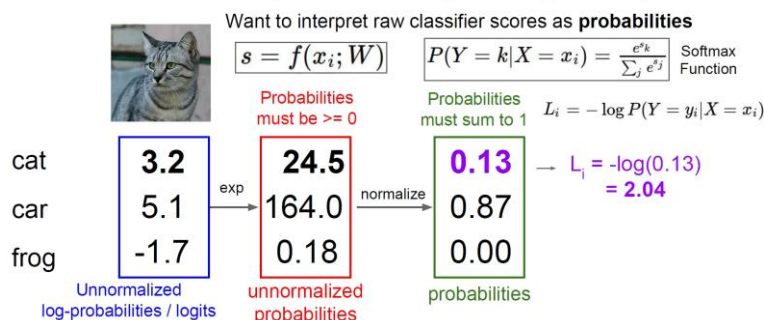
U oba slučaja, regularizacioni član se dodaje kao dodatni član u funkciju greške koja se optimizuje pri treniranju modela. Računanje regularizacionog člana služi da balansira izbor parametara modela između minimizacije greške na trening skupu i minimizacije overfitting-a.

5. [DL01c_Loss_Functions_and_Optimization] Šta predstavlja i kako se računa izlaz softmax sloja? Kako se na osnovu njega računa funkcija greške (engl. cross-entropy loss).

Ideja je da na izlazu mreže dobijemo **verovatnoću** koja klasa je najbolja. Vrš se normalizacija svih verovatnoća, zbir normalizovanih verovatnoća uvek daje 1 (statistika...).

Loss je (negativni) logaritam za onu klasu koju znamo da je tačna (pošto je logaritam, kreće se od 0 do beskonačnosti).

Softmax Classifier (Multinomial Logistic Regression)



Malo lepše:

Softmax sloj se koristi na izlazu neuralnih mreža za zadatke klasifikacije sa više klasa (klasifikacija višeklasnih problema). On transformiše sirove izlaze (logite) modela u verovatnoće. Svaka verovatnoća predstavlja koliko je model siguran da uzorak pripada određenoj klasi, pri čemu su sve verovatnoće pozitivne i sabiraju se do 1.

Kako se računa:

Eksponecijalna funkcija: Za svaki logit (izlaz iz modela) izračunava se njegova ekspancijalna vrednost.

Normalizacija: Svaka ekspancijalna vrednost se deli sa sumom svih ekspancijalnih vrednosti logita, što osigurava da se verovatnoće sabiraju do 1.

Kako se na osnovu softmax izlaza računa funkcija greške (cross-entropy loss)?

Cross-entropy loss meri koliko dobro model predviđa stvarne klase. Ona kvantifikuje razliku između predviđenih verovatnoća (iz softmax sloja) i stvarnih etiketa. Računa se kao negativni logaritam verovatnoće.

Kako se računa:

Uzmi verovatnoće predviđene softmax slojem za svaku klasu.

Uporedi ih sa stvarnim etiketama: Ako je stvarna klasa 1 (što znači da uzorak pripada toj klasi), uzima se negativan logaritam predviđene verovatnoće za tu klasu.

Averaging (prosečna vrednost): Ako se radi o više uzoraka, računa se prosečna vrednost ovih vrednosti za sve uzorke.

Commented [1]: Mislim da je samo ovako nešto dovoljno za loss

Commented [2]: fali loss

6. [DL01c_Loss_Functions_and_Optimization] Na koji način se vrši obučavanje (promena

parametara) metodom gradijentnog spusta (engl. gradient descent method)? Šta predstavlja stohastički gradijentni spust (engl. Stochastic Gradient Descent - SGD).

“Zamislite da se spuštate niz planinu vezanih očiju. Kako ćete da se spustite sa planine? Opipavanjem terena.”

Obučavanje modela mašinskog učenja **metodom gradijentnog spusta** podrazumeva iterativno podešavanje i promenu parametara modela (težina i pristrasnosti), kako bi se minimizovala funkcija gubitka (loss function). Funkcija gubitka meri koliko dobro model predviđa ciljne vrednosti.

Cilj je pronaći vrednosti parametara koje minimizuju funkciju gubitka.

Jedna metoda - malo promeniti svaki parametar, izračunati funkciju: (ovo je jako sporo i glupo)

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

U praksi se uvek koristi analitički gradijent: (brže, egzaktnije, mnogo bolje)

$$\begin{aligned} L &= \frac{1}{N} \sum_{i=1}^N L_i + \sum_k W_k^2 \\ L_i &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\ s &= f(x; W) = Wx \\ \text{want } \nabla_W L \end{aligned}$$

Idealno bi bilo da se propusti svaka slika, ali malo je nerealno, mnogo je sporo.

Stohastički gradijentni spust je varijanta gradijentnog spusta koja se koristi za ubrzavanje obučavanja modela, posebno kada se radi sa velikim skupovima podataka. U klasičnom gradijentnom spustu (batch gradient descent), gradijent se izračunava korišćenjem celokupnog skupa podataka, što može biti veoma sporo i neefikasno za velike skupove podataka. Umesto da se propuste sve slike, propusti se neki mini batch (32/64/128, ali u praksi veličina koja može da stane na grafičku), propusti se taj batch, izvrši se promena parametara za taj batch, unapredi se znak, pa se propusti naredni itd.

$$\begin{aligned} L(W) &= \frac{1}{N} \sum_{i=1}^N L_i(x_i, y_i, W) + \lambda R(W) \\ \nabla_W L(W) &= \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W) \end{aligned}$$

7. [DL02_Neural_Networks_and_Backpropagation] Napisati matematičku funkciju po kojoj se računa izlaz neuronske mreže sa jednim skrivenim slojem koji koristi ReLU aktivacionu funkciju.

Najprostija neuronska mreža - kombinacija linearnog klasifikatora i RELU aktivacione fje (bias je izostavljen radi uprošćavanja). Aktivaciona fja uvodi nelinearnost (inače bi linearni klasifikator dao linearni izlaz), a NN mora da ima nekakvu nelinearnost.

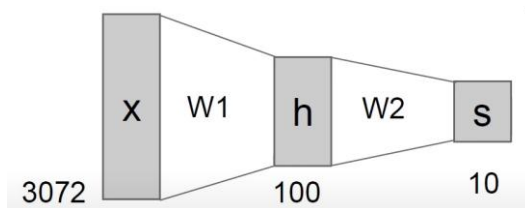
Commented [3]: Novi_parametar = stari_parametar - learning rate * gradijent

Brat moj očekuje da se napiše izraz sa slike ispod za 2-layer NN:

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

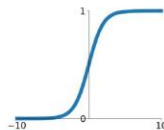
$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$



8. [DL02_Neural_Networks_and_Backpropagation] Nacrtati grafike sledećih aktivacionih funkcija: sigmoidalna, tangens hiperbolički, ReLU, Leaky ReLU i ELU.

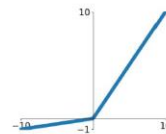
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



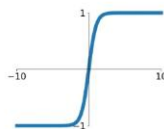
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

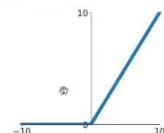


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

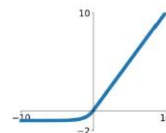
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



9. [DL02_Neural_Networks_and_Backpropagation] Nacrtati graf izračunavanja (engl. computational graph) za funkciju $f = (x + y)z$ i odrediti parcijalne izvode $\partial f / \partial x$, $\partial f / \partial y$, $\partial f / \partial z$, kada su vrednost ulaza $x = 3$, $y = -2$, $z = 5$.

ČU NACRTAM NOVU

Backpropagation: a simple example

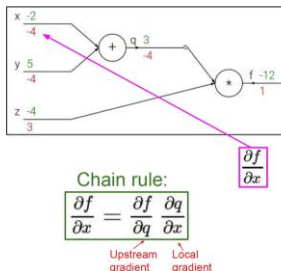
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

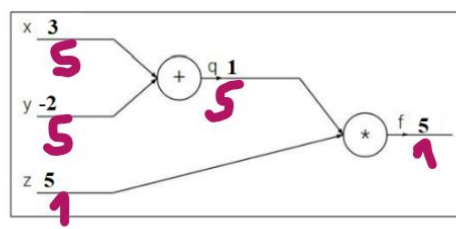


Naš slučaj:

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

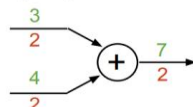
e.g. $x = 3, y = -2, z = 5$



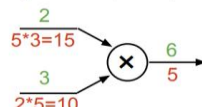
$$\frac{\partial f}{\partial x} = 5, \frac{\partial f}{\partial y} = 5, \frac{\partial f}{\partial z} = 1$$

10. [DL02_Neural_Networks_and_Backpropagation] Grafički ilustrovati četiri šablona toka gradijenata (engl. patterns in gradient flow).

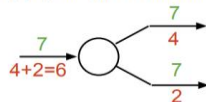
add gate: gradient distributor



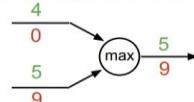
mul gate: "swap multiplier"



copy gate: gradient adder

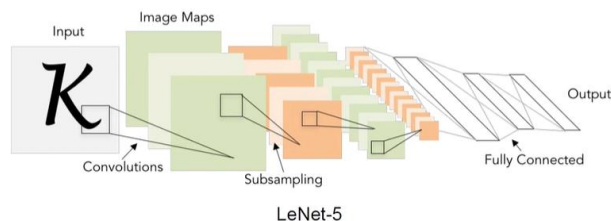


max gate: gradient router



11. [DL03_Convolutional_Neural_Networks] Grafički predstaviti i opisati arhitekture LeNet5 mreže. Koji sve tipovi slojeva se u njoj pojavljuju i koja je njihova uloga?

LeNet-5 je klasifikacioni model konstruisan za prepoznavanje rukom pisanih cifara (za pošiljke). Razvio ga je Yann LeCun 1998. godine i bio je jedan od prvih uspešnih CNN modela za prepoznavanje uzoraka. Arhitektura LeNet-5 sastoji se od nekoliko slojeva, uključujući konvolucione slojeve, slojeve sažimanja (pooling) i potpuno povezane slojeve.



ChatGPT (pošto on ništa nije reko za slojeve):

Konvolutivni sloj (Convolutional Layer):

Ulaz: Slika rukom pisane cifre (obično dimenzija 32x32 piksela).

Osnovna uloga: Detektovanje lokalnih fičera poput ivica i oblika, prosleđivanje mapa dalje kroz mrežu.

Karakteristike: Konvolutivni filteri primenjuju se na ulazni sloj kako bi se generisali mapiranja fičera. Tipično se koriste ReLU aktivacione funkcije nakon konvolucije radi uvođenja nelinearnosti.

Sloj sažimanja (Pooling Layer):

Ulaz: Izlaz konvolutivnog sloja.

Osnovna uloga: Smanjenje dimenzionalnosti i broja parametara konvolutivnog sloja, kao i postizanje lokalne translacione invarijantnosti.

Karakteristike: Tipično se koristi Max Pooling, gde se uzima maksimalna vrednost u određenom regionu. Nekoliko konvolutivnih slojeva može se ponavljati sa slojevima sažimanja kako bi se postigla postepena redukcija dimenzionalnosti.

Potpuno povezani sloj (Fully Connected Layer):

Ulaz: Izlaz sloja sažimanja (u ravnom obliku).

Osnovna uloga: Klasifikacija izlaznih fičera.

Karakteristike: Svaki neuron u ovom sloju povezan je sa svakim neuronima prethodnog sloja.

Koristi se Softmax aktivaciona funkcija za generisanje verovatnoća pripadnosti različitim klasama.

Izlazni sloj:

Ulaz: Izlaz potpuno povezanog sloja.

Osnovna uloga: Dodeljivanje verovatnoća različitim klasama.

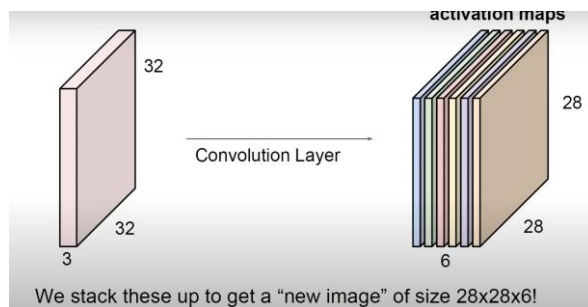
Karakteristike: Broj neurona u izlaznom sloju odgovara broju klasa koje se predviđaju.

Softmax funkcija se koristi za generisanje verovatnoća pripadnosti svakoj klasi.

12. [DL03_Convolutional_Neural_Networks] Koja je uloga i na koji način se formira izlaz konvolutivnog sloja kod konvolutivnih neuronskih mreža (ilustrovati na primeru)? Kolika se mapa na izlazu dobija ukoliko konvolutivni sloj sadrži 6 filtara 5x5x3, a na ulazu se dovede slika dimenzija 32x32x3?

Kao ulaz u konvolucioni sloj, koristi se direktno 3D struktura, nema flattening-a, u startu slika ima 3 kanala (RGB). Uzima se filter manjih dimenzija (treća dimenzija mora da se poklapa, tj. da bude 3), jednim filtrom se dobija jedna ulazna slika (jednim filtrom dobijamo jedan kanal), što nije dovoljno. Treba definisati odgovarajući broj filtera koji daju odgovarajući broj aktivacionih mapa.

Dakle, koliko para - tolko muzike (koliko filtara - tolko mapa), za 6 filtara dimenzija 5x5x3, dobilo bi se 6 aktivacionih mapa (dimenzija 28x28x3).



Malo lepše rečeno:

Ulaz u konvolucioni sloj može biti slika ili rezultat prethodnog sloja. Recimo da imamo ulaznu sliku dimenzija 32x32x3 (RGB slika).

Filteri (Kernel): Konvolucioni sloj primenjuje set filtera (jezgra) na ulaz. Filteri su obično manjih dimenzija od ulaza, npr. 5x5x3. Svaki filter ima povezane težine koje se nauče tokom obuke.

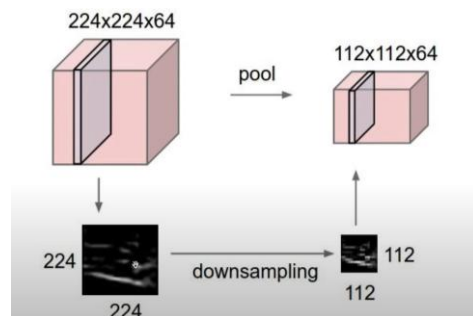
Konvolucija: Filter se pomera preko ulazne slike (sa određenim korakom - stride), i na svakom položaju računa se skalarni proizvod između filtera i odgovarajuće regije ulazne slike. Rezultati se kombinuju kako bi se formirao izlazni piksel.

Aktivaciona funkcija: Nakon konvolucije, na rezultat se primenjuje aktivaciona funkcija kao što je ReLU (Rectified Linear Unit), koja uvodi nelinearnost u model.

Mapiranja fičera (Feature Maps): Konačni izlaz su mape fičera koje sadrže odgovarajuće karakteristike iz ulazne slike.

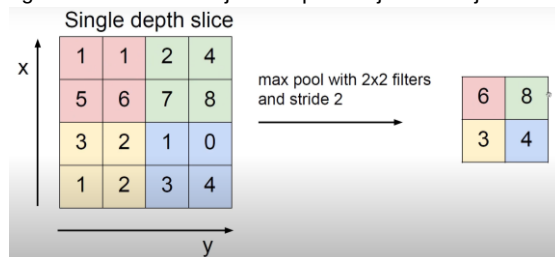
13. [DL03_Convolutional_Neural_Networks] Koja je uloga i na koji način se formira izlaz pooling sloja kod konvolucionih neuronskih mreža (ilustrovati na primeru)? Kolika se mapa na izlazu dobija ukoliko se na ulaz max pooling sloja veličine kernela 3x3 sa pomeranjem (engl. stride) 2, dovede mapa dimenzija 32x32x3?

Pooling sloj radi smanjivanje prostorne dimenzije. Iz **prostornog domena sada se prelazi u domen fičera**. Poenta je da se prostorna dimenzija smanji, kako bi se što bolje došlo do glavnog sadržaja slike (operacijom max ili avg).



Zašto max obično ima najviše smisla? Hoćemo da pronađemo vrednost koja odudara, sadrži nešto važno, najčešće je to max.

Primer za max pooling sa filterom dimenzija 2x2 i pomerajem 2 dat je u nastavku:



$$\text{Dimenzija izlaza} = \left\lfloor \frac{\text{Dimenzija ulaza} - \text{Veličina kernela}}{\text{Korak}} \right\rfloor + 1$$

Neka generalna formula je:

Odgovor:

U ovom slučaju, ako je ulazna mapa dimenzija 32x32x3 i koristi se max pooling sloj veličine kernela 3x3 sa pomeranjem 2, izlazna mapa će biti dimenzija:

visina izlaza = (visina ulaza - visina kernela) / pomeranje + 1 = (32 - 3) / 2 + 1 = 15

širina izlaza = (širina ulaza - širina kernela) / pomeranje + 1 = (32 - 3) / 2 + 1 = 15

dubina izlaza = dubina ulaza = 3

Tako da će izlazna mapa biti dimenzija 15x15x3. Mapa će biti smanjena na 1/2 u obe dimenzije (15 = 32/2), dok će dubina ostati ista.

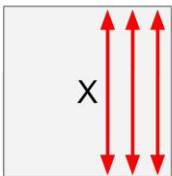
15x15x3 (ako se nacrtu uklapa se)

14. [DL04_Training_Neural_Networks_I] Zbog čega se koristi i na koji način se sprovodi batch normalizacija aktivacionih mapa.

Commented [4]: Treba li ova matematika?

(39. minut) Osnovna ideja - **Batch normalizacija je tehnika koja se koristi za poboljšanje treniranja dubokih neuronskih mreža. Cilj batch normalizacije je stabilizacija i ubrzanje obuke mreže normalizacijom ulaza svakog sloja.** Izvršiti normalizaciju aktivacija iz prethodnih slojeva, tako da bude ista varijansa i da budu centrirani na 0.

Input: $x : N \times D$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \text{Per-channel mean, shape is D}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \text{Per-channel var, shape is D}$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \quad \text{Normalized x, Shape is N x D}$$

D

Šta ako ne želimo uvek da ulazi budu centrirani na 0 i da bude ista varijansa? Uvodimo parametre gama, beta (learnable su, mogu da se obučavaju). Ako obučavanje zahteva da ne koristimo batch normalizaciju, to može da se kompenzuje korišćenjem game i bete.

Input: $x : N \times D$

Learnable scale and shift parameters:
 $\gamma, \beta : D$

Learning $\gamma = \sigma$,
 $\beta = \mu$ will recover the identity function!

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \text{Per-channel mean, shape is D}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \text{Per-channel var, shape is D}$$

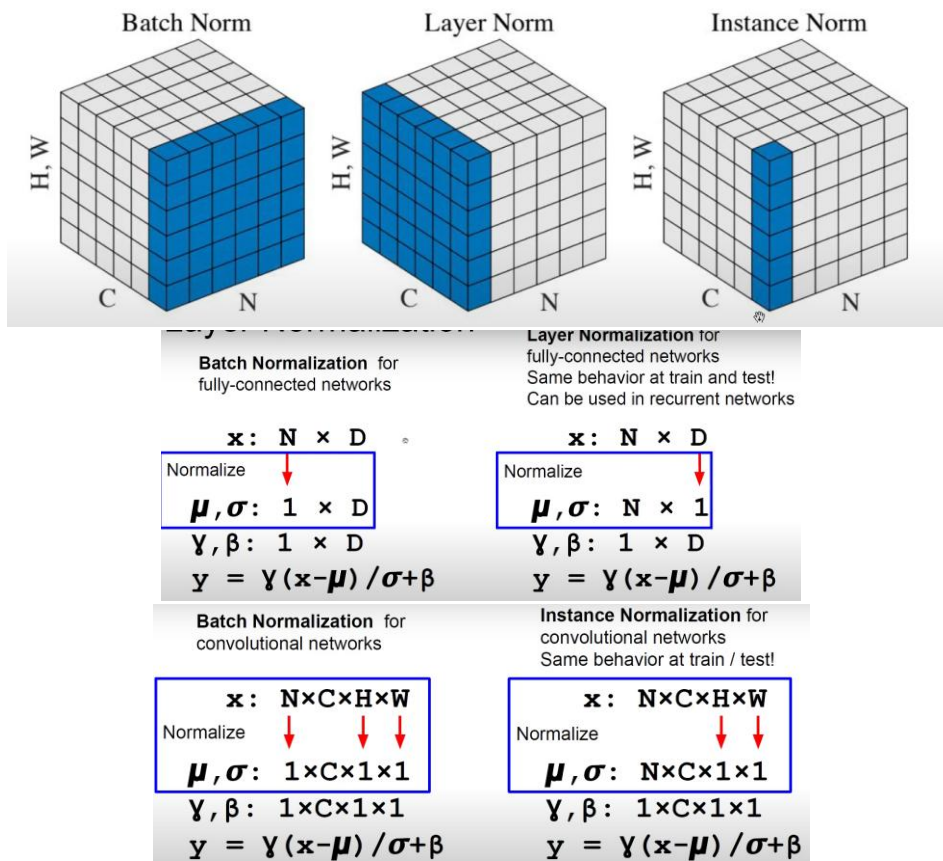
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \quad \text{Normalized x, Shape is N x D}$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j \quad \text{Output, Shape is N x D}$$

Tipično se BN slojevi stavljaju između konvolucionih slojeva i aktivacione funkcije.

15. [DL04_Training_Neural_Networks_I] Koje sve vrste normalizacija postoje. Grafički predstaviti šta koja od njih normalizuje?

Postoje batch normalizacija, layer normalizacija i instance normalizacija.



Layer - obuhvata sve neurone u jednom sloju
Batch - svaki mini-batch posebno

Malo lepše:

1. Batch Normalizacija (Batch Normalization)

Normalizuje podatke unutar mini-batch-a za svaki feature, tokom treniranja neuronske mreže.

Grafički: Normalizacija po kolonama unutar mini-batch-a.

2. Layer Normalizacija (Layer Normalization)

Normalizuje podatke unutar istog sloja za svaki uzorak pojedinačno, nezavisno od ostalih uzoraka u mini-batch-u.

Grafički: Normalizacija po redovima (svakom uzorku) unutar sloja.

3. Instance Normalizacija (Instance Normalization)

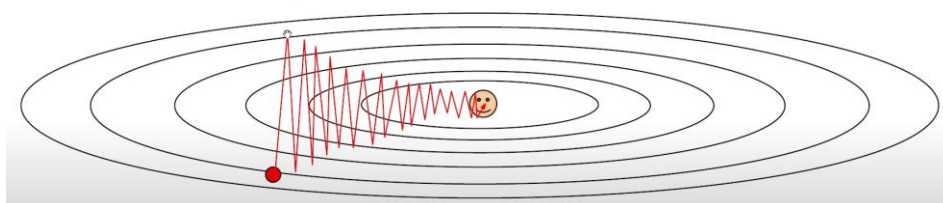
Normalizuje podatke po svakoj instanci (svakom uzorku) nezavisno, obično se koristi u zadacima generisanja slika.

Grafički: Normalizacija po svakom uzorku unutar mini-batch-a, nezavisno od slojeva.

16. [DL05_Training_Neural_Networks_II] Koji su problemi kod stohastičkog gradijentnog spusta (engl. SGD) i kako se prevazilaze korišćenjem momenta?

Šta ako se loss menja brzo u jednom pravcu, a sporo u drugom? Onda se javlja efekat prikazan na slici. Čik cak silazak po jednoj osi i jako ažuriranje, po drugoj slabo.

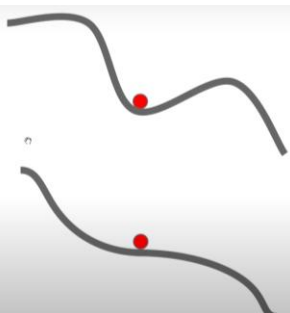
Very slow progress along shallow dimension, jitter along steep direction



Šta ako loss fja ima lokalni minimum (model se zaglavi u toj tački) ili saddle point (usporenu konvergenciju)? Još problema...

What if the loss function has a **local minima** or **saddle point**?

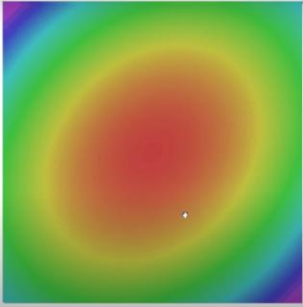
Zero gradient, gradient descent gets stuck



Lutanje, umesto da se ide pravo do cilja, gradijent luta (nije nacrtano na slici)

Our gradients come from minibatches so they can be noisy!

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(x_i, y_i, W)$$

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W)$$


Ideja sa momentumom: Stalno se akumulira kako su se menjali parametri i taj koeficijent se koristi pri novim akumuliranjima.

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
    dx = compute_gradient(x)
    x -= learning_rate * dx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
    dx = compute_gradient(x)
    vx = rho * vx + dx
    x -= learning_rate * vx
```

Malo lepše:

1. Sporo konvergiranje:

Problem: SGD može sporo konvergirati prema optimumu jer ažurira težine na osnovu svakog pojedinačnog uzorka, što može uzrokovati fluktuacije i sporo napredovanje prema minimumu funkcije greške.

2. Zaglavljivanje u lokalnim minimumima:

Problem: SGD može zapeti u lokalnim minimumima funkcije greške, posebno u složenim funkcijama sa mnogo minimuma.

3. Nestabilna putanja optimizacije:

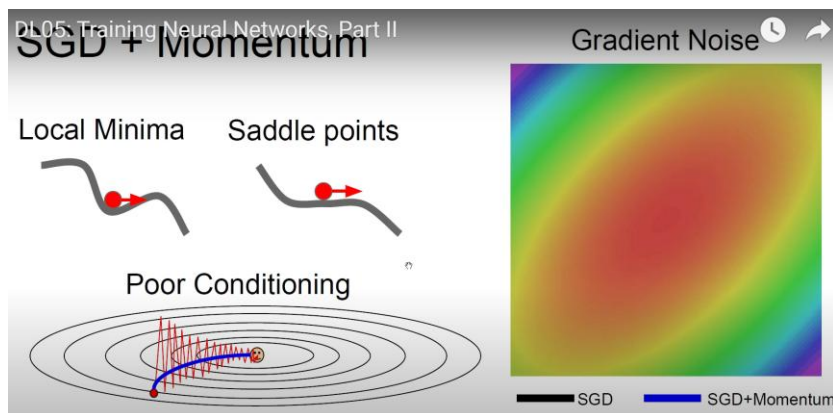
Problem: Zbog velike varijabilnosti u koracima optimizacije, putanja može biti vrlo nestabilna i teško je kontrolisati.

Korišćenje momenta za prevazilaženje problema:

Moment je tehnika koja se koristi za ubrzavanje konvergencije i stabilizaciju putanje optimizacije.

Kako radi moment:

1. **Akumulacija gradijenata:** Umesto da se težine ažuriraju direktno na osnovu trenutnog gradijenta, moment akumulira gradijente kroz vreme.
2. **Smanjenje fluktuacija:** Ova akumulacija pomaže da se smanje fluktuacije i "zaglađuje" putanja optimizacije, što vodi bržem i stabilnijem konvergiranju.



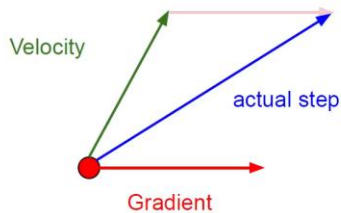
17. [DL05_Training_Neural_Networks_II] U čemu je specifičnost Nestorovog momenta u odnosu na standardni moment i zbog čega može da da bolje rezultate?

Standardni moment ažurira parametre na osnovu trenutnog gradijenta i akumuliranog momenta. Nesterov moment unapred uzima u obzir "ubrzanje" parametara pre nego što izračuna gradijent. Drugim rečima, izračunava gradijent ne u trenutnoj tački, već u tački gde bi parametri bili nakon primene momenta.

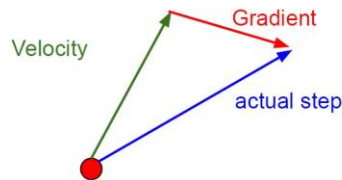
ChatGPT: Intuitivno objašnjenje

Zamislite da ste na vrhu brda i želite da se spustite u dolinu. Sa standardnim momentom, idete u pravcu trenutnog nagiba, koristeći deo prethodnog kretanja da biste ubrzali. Sa Nesterovim momentom, pre nego što napravite korak, prvo pogledate malo napred da biste videli gde bi vas trenutna brzina odvela, i koristite tu informaciju da napravite pametniji korak. Ovaj "pogled unapred" omogućava efikasnije kretanje ka cilju.

Momentum update:



Nesterov Momentum



Malo lepše:

Nesterov moment je modifikacija standardnog momenta koja se koristi u algoritmima optimizacije, posebno u kontekstu stohastičkog gradijentnog spusta. Specifičnost Nestorovog momenta je da uzima u obzir gradijent kako bi prilagođavao parametre mreže.

Kod standardnog momenta, trenutni gradijent se kombinuje sa prethodnim korakom ažuriranja, pri čemu se parametri mreže pomnoženi prethodnim korakom kretanja "momentumom" dodaju trenutnom gradijentu. Ovo pomaže u ubrzanju konvergencije algoritma optimizacije, ali može dovesti do prevelikih koraka u situacijama kada se aproksimacija gradijenta u blizini minimuma pokaže netačnom.

Nesterov moment rešava ovaj problem tako što prvo koristi prethodni korak ažuriranja da "predvidi" sledeći korak i tada izračunava trenutni gradijent. Na osnovu tog predviđanja, ažurira se parametar mreže. Ova tehnika omogućava da se izbegnu preveliki koraci u blizini minimuma i poboljšava konvergenciju.

Zbog ove specifičnosti, Nesterov moment može da pruži bolje rezultate od standardnog momenta, posebno u situacijama kada postoji velika varijacija u gradijentima ili kada se aproksimacija gradijenta pokazuje netačnom u blizini minimuma.

18. [DL05_Training_Neural_Networks_II] Navesti koji sve algoritmi za optimizaciju se koriste i koji se smatra najboljim (najbržim)?

1. SGD + Momentum

Nećemo se ponavljati, pišite prethodno pitanje

2. AdaGrad

Ne koristi se više, ali ideja je da se računaju gradijenti, i onda se kvadrat gradijenata dodaje u nekakvu akumuliranu težinu, pa se ažuriranje vrši tako što se podeli težinom akumuliranih gradijenata - efekat je da ste stalno smanjuje LR, u nekom trenutku će da stane obučavanje (loše je što se možda mreža ne obuči dok dodjemo do trenutka da LR prestane da se menja)

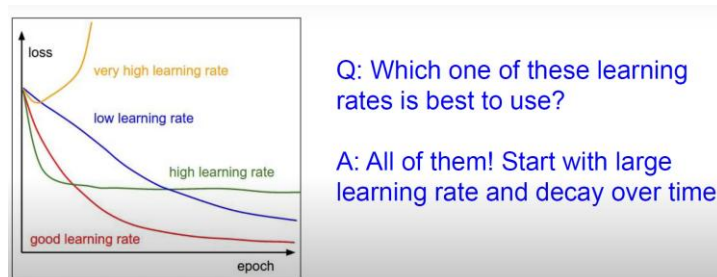
3. RMSProp

Malo promenjen AdaGrad, ubacuje se neki `decay_rate` koji množi staru vrednost, povremeno se smanjuje ta akumulirana težina

4. Adam

Dodaje se neki dodatni član, koji se koristi da smanji moment

(Bože me sačuvaj kako je objašnjeno)



Žuti - mnogo veliki LR, ne obučava

Zeleni - nije loš, ali brzo dođe do zasićenja

Plavi - mnogo nizak LR za start

Crveni - dobar

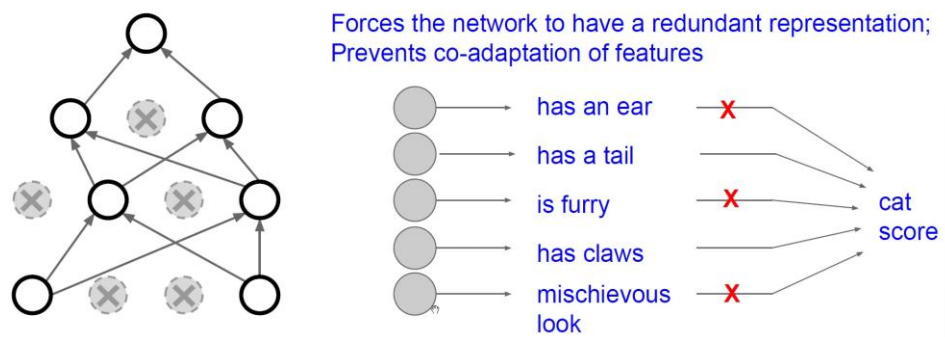
Malo lepše:

Postoji nekoliko popularnih algoritama za optimizaciju kod treniranja neuronskih mreža. Evo nekoliko od njih:

1. Stohastički gradijentni spust (Stochastic Gradient Descent, SGD): To je osnovni algoritam optimizacije koji se često koristi za treniranje neuronskih mreža. On minimizira funkciju gubitka prilagođavanjem težina mreže koristeći gradijent funkcije gubitka u odnosu na težine. Stohastički se naziva jer se gradijent računa na temelju malih podskupova (serija) podataka umesto celog skupa podataka.
2. ADAM (Adaptive Moment Estimation): Ovo je popularan optimizacijski algoritam koji kombinuje prednosti stohastičkog gradijentnog spusta i RMSprop algoritma. ADAM održava adaptivne stope učenja za različite parametre na temelju **prvog i drugog momenta gradijenta**.
3. RMSprop (Root Mean Square Propagation): RMSprop takođe prilagođava stope učenja za svaki parametar mreže, ali koristi samo prvi moment gradijenta. Takođe koristi pokretni **prosek kvadrata gradijenta** kako bi normalizovao stope učenja.
4. Adagrad (Adaptive Gradient Algorithm): Adagrad prilagođava stope učenja za svaki parametar na temelju **kvadrata akumuliranih gradijenata**. To znači da parametri koji imaju veći gradijent dobijaju manje stope učenja tokom vremena.

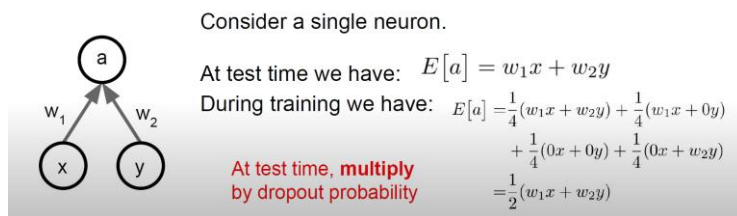
19. [DL05_Training_Neural_Networks_II] Objasniti tehniku odbacivanja (engl. dropout) i navesti zašto se koristi. Na koji način se vrši u toku obučavanja, a kako kasnije u toku eksploatacije mreže?

U fazi treniranja (obučavanja), sa određenom verovatnoćom određene aktivacije postavljamo na 0. Ideja - ubiti neke neurone, u svakom ciklusu dobijamo nekakav drugačiji model koji se obučava. Kao da je efekat da svaki put isprobavamo drugačiji model, što povećava robusnost. Dropout čini da izlaz bude random.



Want to approximate the integral

$$y = f(x) = E_z[f(x, z)] = \int p(z)f(x, z)dz$$



Kasnije - u toku testiranja nema množenje nulom, hoćemo da smanjimo randomness, aktivacije se množe dropout verovatnoćom kako bi se smanjio srednji izlaz.

Malo lepše:

Dropout je tehnika koja se koristi u dubokom učenju za **borbu protiv prenaučivosti (overfittinga) i poboljšanje generalizacije modela**. Ideja je da se slučajno isključe određeni neuroni tokom treniranja, što prisiljava mrežu da nauči robusniju i generalizovaniju reprezentaciju podataka.

Tokom treniranja, svaki neuron unutar sloja ima verovatnoću P da će biti isključen (dropout), pri čemu njegov izlaz postaje 0. Ovo sprečava prenaučivost jer ostali neuroni moraju preuzeti odgovornost za obradu podataka.

Tokom testiranja, svi neuroni su aktivirani, ali izlazi se skaliraju faktorom $(1 - p)$ kako bi se održala očekivana vrednost izlaza (p je verovatnoća isključenja neurona tokom procesa treniranja). Dropout tehnika pomaže u poboljšanju generalizacije i robusnosti modela, uz varijacije koje zavise od arhitekture i zadatka dubokog učenja.

20. [DL05_Training_Neural_Networks_II] Objasniti tehniku proširivanja podataka (engl. Data augmentation) i navesti zašto se koristi. Na koji način se vrši u toku obučavanja, a kako kasnije u toku eksploatacije mreže?

U fazi treniranja ne koristiti samo slike koje imamo, već izvršiti i neakve promene podataka, kako bi se dobio mnogo veći ulazni skup. Korisno, jer jako često nemamo toliko podataka u skupu, pa je ovo dosta korisno. (flipni mačku) - tokom obučavanja: translacija, rotacija, skaliranje, strečing, refleksija, šum, random kropovanje... Tokom eksploatacije se obično ne koristi, želimo originalne ulazne podatke.

Malo lepše:

Data augmentation je tehnika koja se koristi za veštačko povećanje veličine skupa podataka za obučavanje putem generisanja novih podataka iz postojećih podataka. Ovo se postiže primenom različitih transformacija na originalne podatke.

Zašto se koristi?

Povećanje raznovrsnosti podataka: Omogućava stvaranje više varijacija postojećih uzoraka, čime se poboljšava sposobnost modela da generalizuje na nove, neviđene podatke.

Smanjenje prenaučavanja (overfitting): Dodavanjem varijacija, model postaje manje sklon prilagođavanju šumovima ili specifičnostima trening podataka.

Kako se vrši tokom obučavanja?

Tokom treniranja, data augmentation se primenjuje u realnom vremenu, generišući nove varijacije podataka za svaki mini-batch. Neke uobičajene tehnike proširivanja podataka uključuju:

Rotacije: Rotiranje slika za određeni stepen.

Translacije: Pomicanje slika horizontalno ili vertikalno.

Skaliranje: Promena veličine slika.

Flipovanje: Horizontalno ili vertikalno okretanje slika.

Šum: Dodavanje nasumičnog šuma.

Promena osvetljenja: Povećanje ili smanjenje osvetljenja slika.

Promena boja: Promena tonova boja ili kontrasta.

21. [DL06_CNN_Architecture] Šta predstavlja i kako se obavlja preneseno učenje (engl. Transfer learning)?

Osnovna pretpostavka je bila da je za korišćenje dubokih NN potrebna velika količina podataka. Ipak ne, tu uvodimo transfer learning. Omogućava da se duboke NN koriste za mnogo manje

skupove podataka. Ako je mali skup, retreniraju se vršni slojevi, dok se za veći skup trenira veći broj slojeva.

	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer	You're in trouble... Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

Malo lepše:

(brt valjda je ovo poenta) Tehnika u mašinskom učenju gde se znanje stečeno prilikom treniranja modela na jednom zadatku koristi za poboljšanje performansi modela na drugom, srodnom zadatku. Umesto da se model trenira od nule, koristi se već trenirani model kao polazna tačka.

Kada radimo sa pretreniranom neuronskom mrežom, pristupi za finetuning modela zavise od količine dostupnih podataka i njihove sličnosti sa originalnim podacima na kojima je mreža trenirana. Evo kako se različite situacije mogu tretirati:

Objašnjenje slike/tabele iznad koju treba da nacrtate:

1. Vrlo sličan dataset i vrlo malo podataka

Pristup: Koristiti linearni klasifikator na gornjem sloju.

Objašnjenje: Ako su novi podaci vrlo slični originalnim podacima na kojima je model treniran, značajke (feature) koje je model naučio će verovatno biti relevantne. Sa vrlo malo podataka, finetuning većeg broja slojeva može dovesti do prenaučavanja. Korišćenje linearnog klasifikatora na gornjem sloju je jednostavan i efektivan način da se iskoriste naučene značajke bez rizika od prenaučavanja.

2. Vrlo različit dataset i vrlo malo podataka

Pristup: Situacija je problematična, pokušajte koristiti linearni klasifikator sa različitih nivoa.

Objašnjenje: Ako su novi podaci vrlo različiti od originalnih, značajke koje je model naučio možda neće biti relevantne. Sa vrlo malo podataka, finetuning je težak jer je rizik od prenaučavanja visok. U ovom slučaju, možete isprobati linearni klasifikator na različitim

nivoima mreže kako biste videli koji nivo najbolje prenosi korisne značajke za nove podatke.

3. Vrlo sličan dataset i dosta podataka

Pristup: Finetuning nekoliko slojeva.

Objašnjenje: Kada imate dovoljno podataka koji su slični originalnim, možete finetuningovati nekoliko gornjih slojeva. Ovo omogućava modelu da prilagodi svoje naučene značajke specifičnostima novog skupa podataka, poboljšavajući performanse bez rizika od prenaučavanja.

4. Vrlo različit dataset i dosta podataka

Pristup: Finetuning većeg broja slojeva.

Objašnjenje: Kada su novi podaci vrlo različiti i imate dovoljno podataka, najbolji pristup je finetuning većeg broja slojeva, ili čak cele mreže. Ovo omogućava modelu da u potpunosti prilagodi svoje značajke novom domenu podataka, pružajući najbolje moguće performanse.

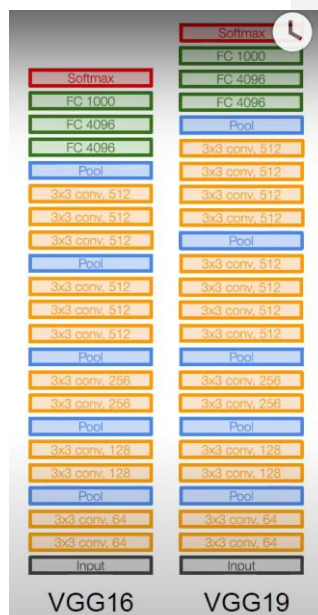
22. [DL06_CNN_Architecture] Koji su osnovni arhitekturni elementi karakteristični za VGGNet?

VGGNet dolazi u nekoliko varijanti, od kojih su najpoznatije VGG16 i VGG19 (19 ima više memorije, samo je malo bolja), koje imaju 16 i 19 slojeva (u smislu konvolucionih i potpuno povezanih slojeva) respektivno. To je znatno više slojeva nego prethodni modeli poput AlexNet-a koji ima 8 slojeva.

Sve konvolucije koriste male filtere dimenzija 3x3, sa stride-om od 1 i padding-om od 1 (što znači da su izlazne dimenzije iste kao ulazne). **SEGA VO MALEČKO FILTERČE**

Max pooling slojevi koriste filtere dimenzija 2x2 sa stride-om od 2, što znači da prepolovljuju dimenzije izlaza. Max pooling se koristi nakon nekoliko konvolucionih slojeva kako bi se smanjile dimenzije i time smanjio broj parametara, a zadržale najvažnije informacije.

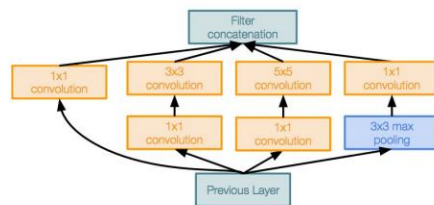
Zašto samo 3x3 konvolucija? Stek od 3 3x3 konv ima isti efekat kao jedna konvolucija dimenzija 7x7, tako da ispada da je ovo čak pametno rešenje



Generalno jedna lepa mreža, lepo se generalizuje mreža za druge zadatke.

23. [DL06_CNN_Architecture] Grafički predstaviti inception modul koji predstavlja osnovu GoogLeNet arhitekture i objasniti ideju koja je tu primenjena.

“Inception module”: design a good local network topology (network within a network) and then stack these modules on top of each other



Inception module

Glavna ideja početnog modula je da se sastoji od višestrukih operacija udruživanja i konvolucije sa različitim veličinama (3×3 , 5×5) paralelno, umesto da se koristi samo jedan filter jedne veličine.

Malo lepše:

Glavna uloga "Inception" sloja je efikasna ekstrakcija fičera iz slika koristeći različite veličine filtara konvolucije unutar istog sloja. Tradicionalno, konvolucijski slojevi koriste jedan fiksni filter za izvlačenje značajki iz ulaza. Međutim, "Inception" sloj kombinuje različite veličine filtara u paralelnim granama i zatim spaja njihove izlaze.

Ova "Inception" struktura omogućuje mreži da nauči fičere na različitim nivoima detalja u istom sloju. Umesto da se oslanja samo na jednu veličinu filtera, "Inception" sloj koristi filtre veličine 1×1 , 3×3 i 5×5 , te Max pooling s filterima veličine 3×3 . Svi ti filteri izvode konvoluciju na ulaznom sloju paralelno.

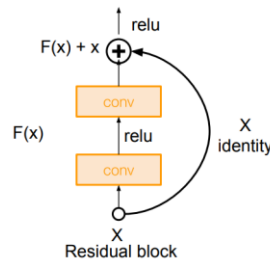
Prednosti "Inception" sloja uključuju:

1. Efikasno korišćenje računarskih resursa: Korišćenjem filtara različitih veličina unutar istog sloja, "Inception" sloj može izvući fičere na različitim nivoima detalja koristeći manje parametara i manje računarskih resursa nego da su svi filteri izvedeni pojedinačno.
2. Poboljšana reprezentacija fičera: Kombinacija različitih veličina filtara omogućuje mreži da nauči različite aspekte slike, uključujući detalje, teksture, rubove i globalne obrasce. To omogućuje mreži da izgradi bogatiju i sveobuhvatniju reprezentaciju slike.
3. Smanjenje dimenzionalnosti: Uz paralelne grane s filterom veličine 1×1 , "Inception" sloj takođe koristi konvoluciju 1×1 za smanjenje dimenzionalnosti pre primene većih filtara. To smanjuje broj kanala fičera i računarske zahteve, što olakšava obradu složenih slika.

24. [DL06_CNN_Architecture] Grafički predstaviti strukturu rezidualnog bloka koji se koristi kod ResNet arhitekture i opisati prednosti njegovog korišćenja.

Very deep networks using residual connections

- 152-layer model for ImageNet
- ILSVRC'15 classification winner (3.57% top 5 error)
- Swept all classification and detection competitions in ILSVRC'15 and COCO'15!



Osnovni gradivni blok ResNet-a je „rezidualni blok“ ili „preskakanje veze“. Ovaj blok omogućava aktivacijama da zaobiđu jedan ili više slojeva preko prečice, što olakšava optimizaciju dubokih mreža.

Koja je polenta ovoga? Uočilo se da se performanse ne poboljšavaju i greške ne smanjuju dodavanjem dodatnih slojeva u mreži (prikazano na slici ispod, ispada da manje greške pravi mreža sa 20 slojeva nego 56). Pomislio bi čovek da je overfittovana, ali nije to problem, ova situacija javlja se na oba skupa - i za treniranje, i za testiranje.

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?



Q: What's strange about these training and test curves?
[Hint: look at the order of the curves]

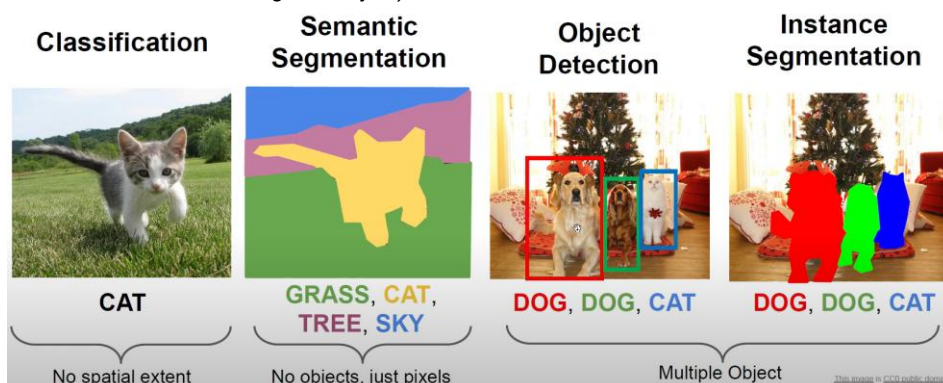
Problem je u obučavanju mreže sa mnogo slojeva, dodavanjem rezidualnih veza, postiže se da gradijent bolje teče (direktno tim vezama), mreža se mnogo lakše obučava i ima bolje performanse.

25. **[DL07 Detection and Segmentation]** Koja su četiri bazična zadatka računarskog vida? Skicirati i opisati ukratko svaki od njih. (2min)

Elementarni zadaci računarskog vida su:

- **Klasifikacija** - mreža treba da da izlaz kojoj klasi pripada ulazna slika, tj šta se na slici nalazi. Ovaj zadatak ne uključuje određivanje bilo kakvih prostornih koordinata, već se slika posmatra kao celina. Na primeru sa slike je na ulazu slika mačke, a na izlazu oznaka klase (u ovom slučaju CAT).

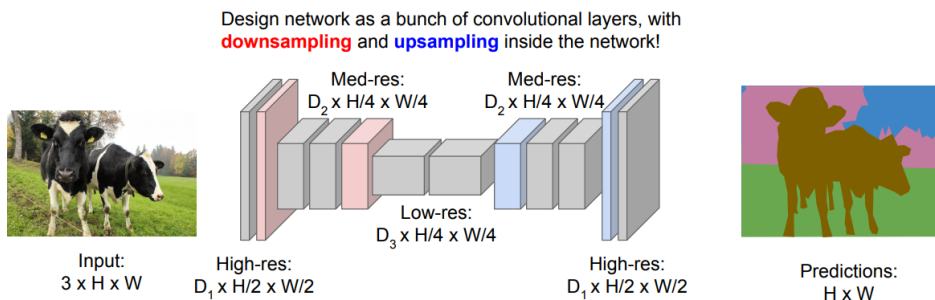
- **Semantička segmentacija** - to je klasifikacija na nivou svakog piksela. Umesto određivanja klase kojoj cela slika pripada, za svaki piksel se posebno određuje čemu on pripada. Na primeru sa slike je slika mačke, a na izlazu oznake za svaki piksel kojoj klasi on pripada (GRASS, CAT, TREE, SKY). Ovakav izlaz daje mnogo više informacija, međutim, na izlazu imamo samo oznake za pojedinačne piksele, ali nemamo nikakve informacije o pojedinačnim objektima (pikseli različitih instanci objekata ne bi bili razdvojeni)
- **Detekcija objekata** - određivanje pravougaonih oblasti gde se pojedinačni objekti nalaze, uz određivanje odgovarajuće labele koja daje informaciju o tome šta se u datoj oblasti nalazi (na primeru sa slike vidimo da su oba psa izdvojeni kao pojedinačni objekti)
- **Segmentacija instanci** - slično semantičkoj segmentaciji, uz dodatak da se vrši razdvajanje pojedinačnih instanci u okviru jedne klase (na primeru sa slike vidimo da su pikseli pojedinačnih instanci pasa razdvojeni iako pripadaju istoj klasi, što ne bi bio slučaj sa semantičkom segmentacijom)



26. [DL07_Detection_and_Segmentation] Na koji način se formiraju potpuno konvolucione mreže za semantičku segmentaciju? Koja dva osnovna elementa one poseduju?

Ova tehnika podrazumeva korišćenje isključivo konvolucionih slojeva, bez korišćenja pooling slojeva, kako bi prostorna dimenzija ostala ista. Ideja kod ove tehnike jeste da se na osnovu mape koja se nađe na izlazu poslednjeg konvolucionog sloja izvrši klasifikacija za svaki piksel kojoj on klasi pripada, pomoću argmax funkcije. **Problem kod ovakvog rešenja jeste da je operacija konvolucije nad velikim slikama veoma skupa i zahteva puno memorije i računarske moći,** pa u većini situacija ovakvo rešenje nije izvodljivo.

Kako bi se ovaj problem izbegao, predložena je arhitektura, koja se sastoji od enkodera (downsampling) i dekodera (upsampling). Downsampling deo predstavlja klasičnu konvolucionu neuronsku mrežu (vrši se smanjenje prostorne dimenzije), a upsampling deo se može shvatiti kao "obrnuti" postupak, gde povećavamo prostornu dimenziju na uštrb broja kanala. Ovo je osnovna ideja za rešenje ovog problema, a u kasnijim radovima je ovakva arhitektura proširena još nekim elementima (npr. Skip konekcije). Povećanje prostorne dimenzije se može vršiti pomoću "unpooling" tehnike, a mnogo češće pomoću transponovane konvolucije.



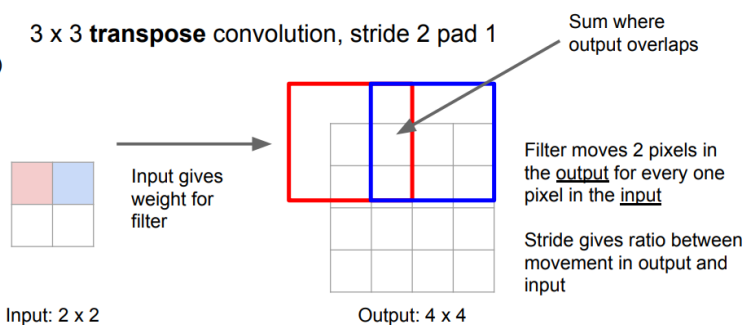
27. [DL07 Detection and Segmentation] Koja je uloga i na koji način se sprovodi transponovana konvolucija (engl. transpose convolution)?

Transponovana konvolucija predstavlja operaciju koja se najčešće obavlja u upsampling delu neuronske mreže i njena uloga je povećanje prostorne dimenzije slike na uštrb broja kanala (za razliku od konvolucije koja povećava broj kanala uz smanjivanje prostorne dimenzije). Normalna konvolucija funkcioniše tako što se odgovarajući filter postavlja preko slike i određuje dot product ulazne slike i filtra, uz pomeranje ovog filtra za određeni stride preko cele slike. Povećanjem stride parametra, možemo uticati na smanjenje prostorne dimenzije izlaza. Transponovana konvolucija se može shvatiti kao "obrnuta" operacija (ulaz je manje prostorne dimenzije od izlaza). **Kod transponovane konvolucije, ulazni piksel daje težine vrednostima u okviru filtra (filter se množi tom vrednošću)**, i odgovarajuće izlazne elemente smeštamo u izlaznu matricu (neki elementi izlaza mogu biti ignorisani ukoliko postoji padding). Slično konvoluciji, ova operacija se obavlja za svaki ulazni piksel, a prilikom postavljanja izlaza filter se pomera za odgovarajući stride, pri čemu se na mestima preklapanja vrši sabiranje. Ovakva operacija omogućava učenje najboljeg načina da se slika poveća po prostornim dimenzijama (uče se vrednosti u okviru filtra).

Other names:

- Deconvolution (bad)
- Upconvolution
- Fractionally strided convolution
- Backward strided convolution

3 x 3 **transpose** convolution, stride 2 pad 1



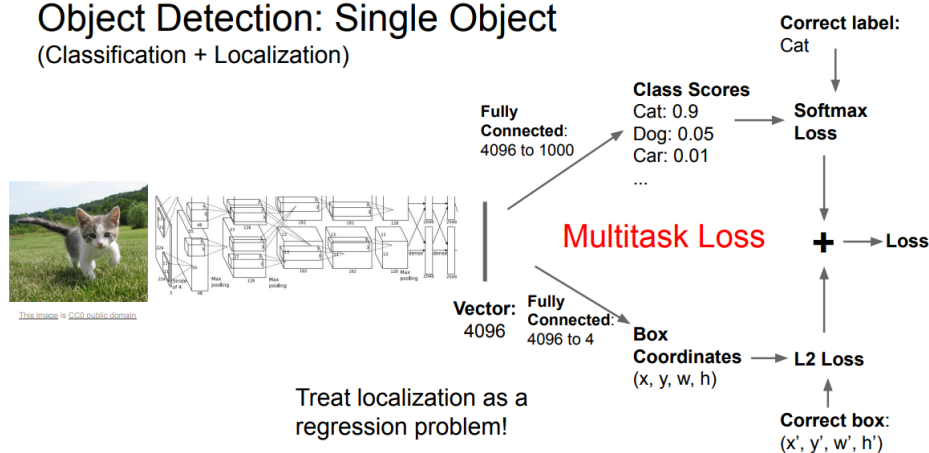
28. [DL07 Detection and Segmentation] Koja je osnovna arhitektura konvolucione mreže za klasifikaciju i lokalizaciju? Grafički ilustrovati i naznačiti razliku u odnosu na konvolucionu mrežu za klasifikaciju.

Ovde je ideja da pored obične klasifikacije dobijemo i neki pravougaonik koji govori o tome u kom delu slike je dati objekat prepoznat (lokalizacija). Ovde je cilj trenirati mrežu koja na izlazu pored klase daje i ovaj bounding box u vidu koordinata. Ovakva arhitektura podrazumeva **postavljanje dve glave na neku neuronsku mrežu**, nakon konvolucionih slojeva i fully connected slojeva (često se koristi gotova i prethodno istrenirana arhitektura nad ImageNet skupom podataka, npr. AlexNet). Uloga neuronske mreže jeste da na svom izlazu da nekakav feature vektor, kreiran na osnovu ulazne slike, a **na osnovu ovog feature vektora se u okviru ovih glava vrši klasifikacija same slike** - prva glava, ali i određivanje koordinata bounding boxa (lokalizacija), što predstavlja problem regresije (kontinualni izlazi). Za treniranje ovakve mreže se koristi softmax loss za glavu koja vrši određivanje klase, a za glavu koja vrši lokalizaciju se koristi loss funkcija kao što je L2 (pogodna za regresiju). Da bismo ovu mrežu obučavali zajednički, definiše se konačan **Multitask loss**, kombinovanjem prethodna dva loss-a. Ovaj konačan loss se najčešće određuje kao težinska suma loss-eva pojedinačnih glava (množe se koeficijentima).

Multitask loss = softmax + L2

Object Detection: Single Object

(Classification + Localization)

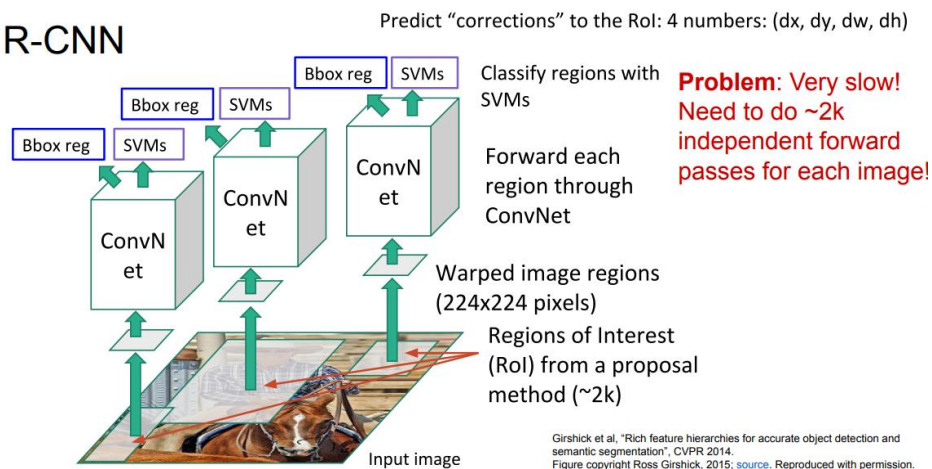


29. [DL07 Detection and Segmentation] Skicirati i objasniti princip rada R-CNN mreže za detekciju objekata. Koji su nedostaci ove arhitekture?

Kako bi se ubrzao ceo postupak, prvi korak kod ovakve detekcije objekata je korišćenje neke pametne logike za pronalaženje kandidata bounding box-eva koji imaju najveće šanse da sadrže neki objekat (umesto da isprobavamo svaki mogući region). Npr Selective Search radi jako brzo i daje 2000 predloga regiona. Rešenje koje koristi ovu tehniku se zove R-CNN (Region-CNN). Prvo se koristi neka od prethodno opisanih tehnika za određivanje RoI (region of interest), koji potencijalno sadrže objekat, u cilju izbegavanja brute force pretrage. Ovi regioni se resize-uju u oblik koji mreža prihvata (koristi se bilo koja mreža, npr. Trenirana nad ImageNet skupom). Ova mreža na svom izlazu za svaki region daje odgovarajuće feature, pri čemu se na osnovu ovih feature-a određuje klasa slike (u originalnom radu je korišćen SVM) i bounding box (regresija), za svaki region).

Najveći nedostatak ovakve arhitekture jeste što je ona dosta spora, jer zahteva propuštanje 2000 regiona pojedinačno kroz neuronsku mrežu.

R-CNN

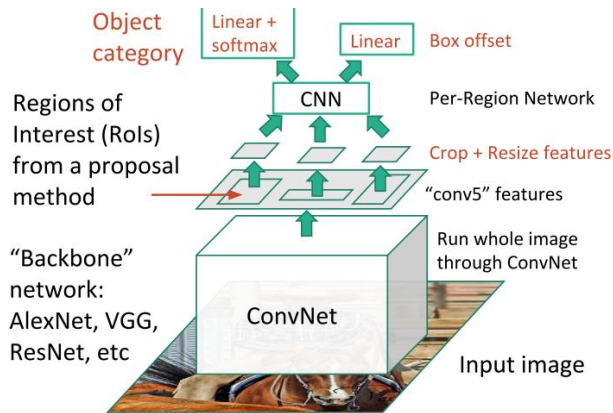


30. [DL07_Detection and Segmentation] Skicirati i objasniti princip rada Fast R-CNN mreže za detekciju objekata. Šta je glavna razlika u odnosu na običnu R-CNN mrežu?

Ideja kod Fast R-CNN: Umesto propuštanja pojedinačnih regiona slike kroz neuronsku mrežu, možemo propustiti celu sliku kroz neuronsku mrežu, a onda vršimo izdvajanje regiona iz neke od mapa fičera.

Funkcioniše na sledeći način: Cela slika se propusti kroz neku backbone mrežu (npr. AlexNet, VGG, ResNet...) i uzima se fičer mapa koja se nalazi na izlazu nekog od konvolucionih slojeva (najčešće pri kraju mreže). Zatim se vrši izdvajanje regiona od interesa (RoI) iz ove fičer mape. RoI u okviru feature mape se određuju tako što se RoI iz originalne slike projektuju na datu feature mapu. Ovi regioni se resize-uju na odgovarajuću dimenziju i zatim se propuštaju kroz neku manju CNN, na osnovu čijeg se izlaza određuje konačna klasa i bounding box. Ovakva arhitektura predstavlja veliko unapređenje u brzini jer se propuštanje kroz veliku mrežu vrši samo jednom. Glavna razlika u odnosu na Slow R-CNN je u tome što se izdvajanje regiona od interesa radi nad fičer mapom (dobijenom korišćenjem neke veće CNN) umesto nad celom slikom, što značajno ubrzava ceo postupak jer klasifikacija i određivanje BB regiona postaje značajno brža.

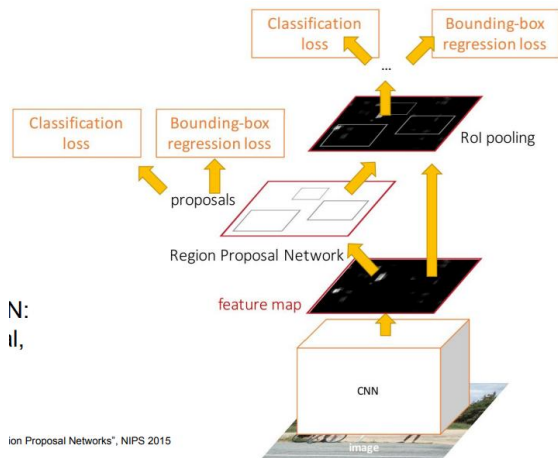
Commented [5]: Je li dovoljno detaljno? Ima više o tome na slajdovima 60-70, ali mislim da to nije deo pitanja.



31. [DL07 Detection and Segmentation] Skicirati i objasniti princip rada Faster R-CNN mreže za detekciju objekata. Šta je glavna razlika u odnosu na Fast R-CNN mrežu?

Faster-R-CNN nastoji da ubrza proces određivanja regiona od interesa, jer taj deo arhitekture oduzima najviše vremena kod Fast R-CNN mreže. U tu svrhu, ova arhitektura uvodi Region Proposal Network (RPN), tj. Mrežu za određivanje predloga za RoI. Ideja je da umesto krošćenja posebnog algoritma iskoristimo neuronsku mrežu, koja se obučava zajedno sa celom mrežom. Arhitektura je gotovo ista kao i kod Fast R-CNN, pri čemu je dodata RPN koja daje predloge regiona. Prilikom treniranja ovakve mreže se koristi kombinovani loss koji se sastoji iz sledeća 4 dela: RPN loss za klasifikaciju da li je u pitanju objekat ili ne, RPN loss regresije koordinata regiona, konačan loss klasifikacije objekata, konačan loss regresije koordinata bounding boxeva objekata.

Commented [6]: Je li treba objašnjenje anchor boxeva sa slajdova 74-79? (Pls ne)



N:
il,

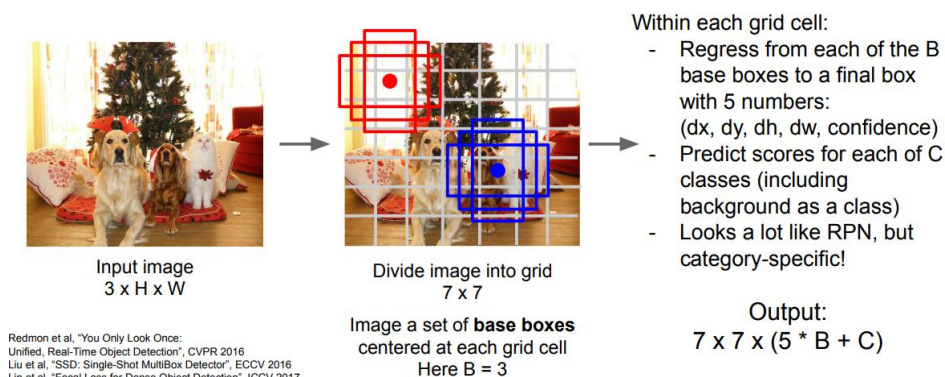
32. [DL07 Detection and Segmentation] Objasniti princip rada jednofaznih detektora objekata (engl. Single-Stage Object Detectors) kao što su YOLO, SSD i RetinaNet. U čemu je glavna razlika u odnosu na Faster R-CNN mrežu?

Jednofazni detektori objekata su jedni od danas najpopularnijih i najkorištenijih detektora objekata i daju najbolje performanse. Jedan od najpopularnijih modela je YOLO (You Only Look Once). Ideja je da se jednim prolazom slike kroz mrežu dobiju izlazi. Ideja je da se **cela slika podeli na neki grid**, i da se **za svaku ćeliju u gridu generiše određeni broj pravougaonika** koji su vezani za tu ćeliju, a zatim se **za svaku ćeliju vrši regresija** u cilju određivanja najboljeg pravougaonika uz neki confidence (koeficijent sigurnosti), zajedno sa predikcijama ocena za svaku klasu. Ukupan broj izlaza je $7 \times 7 \times (5 * B + C)$.

Ovi detektori podsećaju dosta na RPN kod Faster R-CNN-a (regresija za BB, anchored BBs itd.) Za razliku od RPN, ovi detektori vrše klasifikaciju za sve klase objekata (+ pozadina), dok RPN vrši samo klasifikaciju da li je u BB objekat ili ne (category specific).

Za razliku od Faster R-CNN, detekcija se obavlja samo jednim prolazom (single stage). (Kod FasterR-CNN imamo prvo predloge regiona, zatim detekciju i klasifikaciju)

Commented [7]: Je l ovako?? Nije eksplicitno reko, ovo je neki mix sa objašnjenjem chat gpt



33. [DL08 Recurrent Neural Networks] Koja je glavna ideja kod rekurentnih neuronskih mreža koja ih razlikuju od „običnih“ neuronskih mreža? Za koje probleme se ove mreže koriste?

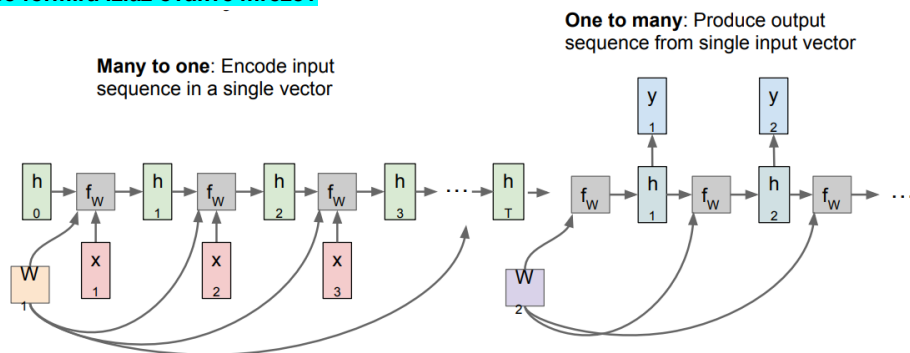
Kod običnih neuronskih mreža imamo ulaze i izlaze fiksne veličine, pri čemu se ulazi provlače kroz niz skrivenih slojeva (npr. Ulaz slika, izlaz skorovi klasifikacije). U nekim situacijama je potrebna veća fleksibilnost kada je u pitanju procesiranje određenih podataka. Upravo ovo se može postići kroišćenjem rekurentnih neuronskih mreža. Neki od problema za koje se RNN mogu koristiti su:

- Image captioning (generisanje opisa slike) - ulaz je slika, izlaz sekvenca reči (one-to-many)
- Klasifikacija sentimenta - ulaz je sekvenca reči, izlaz je ocena sentimenta (many-to-one)
- Prevođenje teksta - i ulaz i izlaz su sekvenca reči (mogu biti različitih dužina) (many-to-many)
- Klasifikacija videa na nivou frejmova - ulaz je sekvenca video frejmova, izlaz je klasifikacija za svaki frejm (many-to-many)

Rekurentne neuronske mreže se mogu shvatiti kao generalna paradigma za rad sa sekvencijalnim podacima čija dužina može varirati. Postoje i posebni slučajevi korišćenja RNN za ulaze/izlaze fiksnih veličina (sekvencijalno porocesanje ne-sekvencijalnih podataka), koji se mogu koristiti za npr. Klasifikaciju (umesto jednog prolaza, mreža više puta “baca pogled” na delove slike), generisanje slika (slika se generiše deo po deo).

Generalno, RNN, je mreža koja ima neki tip unutrašnjeg **skrivenog stanja**, koje se ažurira svaki put kada se na njen ulaz dovedu podaci i od njega zavise izlazi. U svakom trenutku, novo skrieno stanje se računa kao funkcija trenutnog stanja i trenutnih ulaza.

34. [DL08 Recurrent Neural Networks] Skicirati način rada jednostavne rekurentne mreže za prevođenje teksta sa jednim skrivenim slojem i napisati matematičku formulu po kojoj se formira izlaz ovakve mreže?



Ovakve mreže imaju dvodelnu arhitekturu - sastoje se od enkodera i dekodera, pri čemu enkoder prevodi ulaznu sekvencu promenljive dužine u neki vektor (enkoder - many-to-one), a dekode prevodi ovaj vektor u neku izlaznu sekvencu (dekode - one-to-many). h_t predstavlja skriveno stanje mreže i ono se ažurira sa svakim ulazom kao funkcija (koja zavisi od parametara W_a) trenutnog skrivenog stanja i ulaza, a izlazi se formiraju kao funkcija (koja zavisi od parametara W_2) skrivenog stanja.

$$h_t = f_W(h_{t-1}, x_t)$$

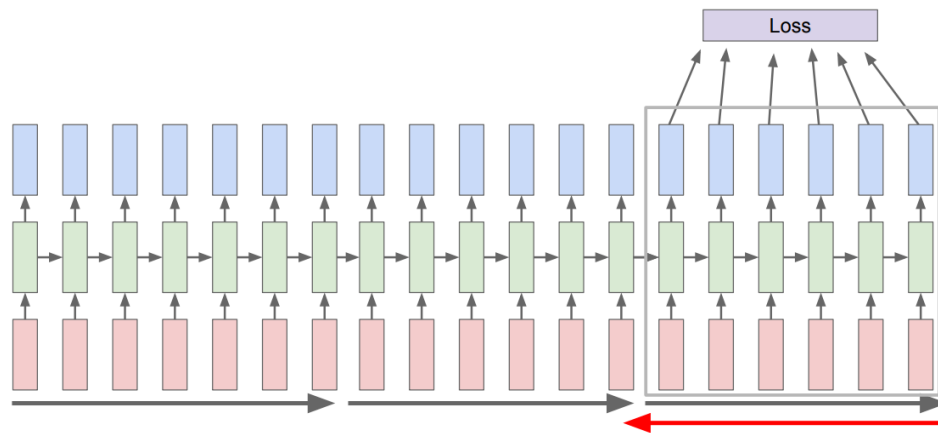
↓

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

35. [DL08 Recurrent Neural Networks] Grafički ilustrovati i objasniti na koji način se vrši obučavanje rekurentnih neuronskih mreža korišćenjem backpropagation algoritma? Kako se naziva ova varijanta backpropagation algoritma?

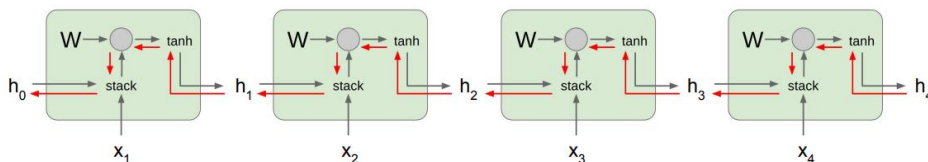
Prilikom obučavanja RNN, klasičan pristup (backpropagation through time) bi podrazumevao da propustimo celu sekvencu (npr. Ceo tekst sa wikipedie) kroz model, a zatim da napravimo prolaz unazad kako bismo izračunali gradijent za samo jedan korak. Ovakav pristup nije upotrebljiv, jer bi radio previše sporo. Zbog ovoga, koristi se varijanta ovog algoritma koja se naziva **truncated backpropagation through time**. Ovde je ideja da, čak iako je ulazna sekvencu veoma dugačka, **propustimo kroz model deo sekvence** (najčešće se koristi 100 kao dužina ove podsekvence tj. chunk-a), **računamo gubitak samo za ovaj deo sekvence** i **onda back-propagiramo i računamo gradijent za ovaj deo**. Za sledeći korak, forward pass se računa kao i obično (imamo skrivena stanja iz prvog prolaza, samo nastavimo sa dovodenjem sekvence na ulaz), a backpropagation se ponovo radi samo nad tim delom sekvence i za nju računamo gradijent). Ovaj postupak podseca na SGD (računamo loss i gradijent za mini-batch-eve).



36. [DL08_Recurrent_Neural_Networks] Koji problemi su karakteristični za obične rekurentne neuronske mreže i koje druge arhitekture su uvedene u cilju prevazilaženja?

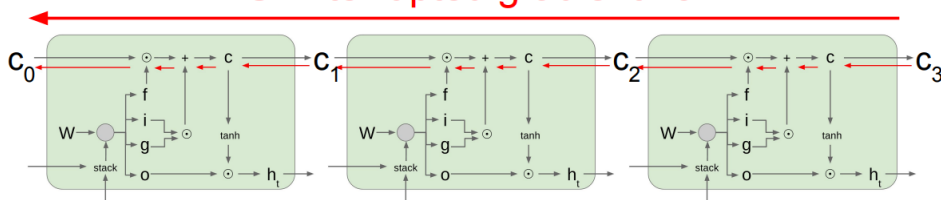
Kod arhitekture običnih (vanilla) RNN-a, u toku procesa backpropagacije, prolazimo kroz korak u kome se vektori stanja i ulaza množe matricom W (matrica W predstavlja parametre funkcije koja na osnovu trenutnog stanja i ulaza računa novo stanje). Prolazak kroz ovaj korak unazad bi podrazumevalo množenje transponovanim faktorima W za svaku ćeliju (svaki deo sekvence). Ovo bi značilo da je za računanje konačnog gradijenta za početno stanje h_0 potreban veliki broj množenja sa faktorima unutar W . Ovaj problem je najbolje posmatrati kroz analogiju množenja skalara. Ako množimo skalar veliki broj puta, njegova vrednost ceprevise porasti ukoliko je on veći od 1, a ukoliko je manji od 1, izgubiće se i postaće blizak nuli (ograničavajući se na apsolutnu vrednost), a jedini način da se ovaj efekat izbegne je da vrednost skalara bude tačno 1. Ako ovaj problem proširimo na matrice, prilikom višestrukog množenja matricom, onda je **potrebno posmatrati najveću singularnu vrednost** - ako je ona veća od 1, gradijent za h_0 postaje prevelik (exploding gradient), a ako je manja od 1, postaje veoma mala (vanishing gradient). Jedan način da se ovo reši je jednostavan gradient clipping - ako je L2 norma gradijenta veća od thresholda, gradijent se clipuje na tu vrednost (koristi se za exploding gradient i često radi ali nije dobro

rešenje). Za rešavanje vanishing gradient problema neophodno je koristiti drugačiju arhitekturu RNN.



Ovo predstavlja modifikaciju klasične RNN - LSTM (Long Short Term Memory), koja ima za cilj da reši problem vanishing i exploding gradijenta (arhitektura LSTM je takva da je tok gradijenta pogodniji). Kod LSTM se čuvaju dva skrivena stanja - h_t (kao kod klasičnih RNN) i pored njega i c_t , koje predstavlja interno stanje. Posledica ovakvog uvođenja internog stanja c_t jeste da tok gradijenta tokom backpropagacije postaje dosta bolji u odnosu na obični RNN i izbegava se višestruko množenje faktorima W i na taj način se izbegavaju problemi sa vanishing i exploding gradijentom.

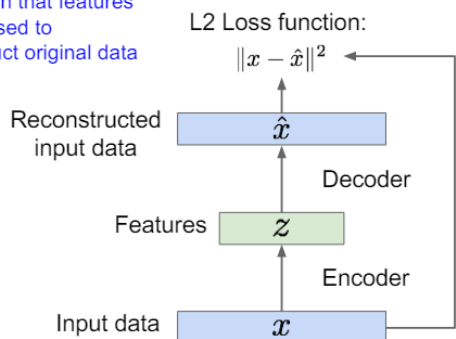
Uninterrupted gradient flow!



Pored LSTM arhitekture, još jedna arhitektura koja rešava ovaj problem je i GRU.

37. [DL09 Generative Models] Grafički ilustrovati i opisati arhitekturu autoenkodera? Na koji način se vrši njihovo obučavanje? Koja funkcija greške (engl. loss) se koristi?

ch that features
used to
reconstruct original data

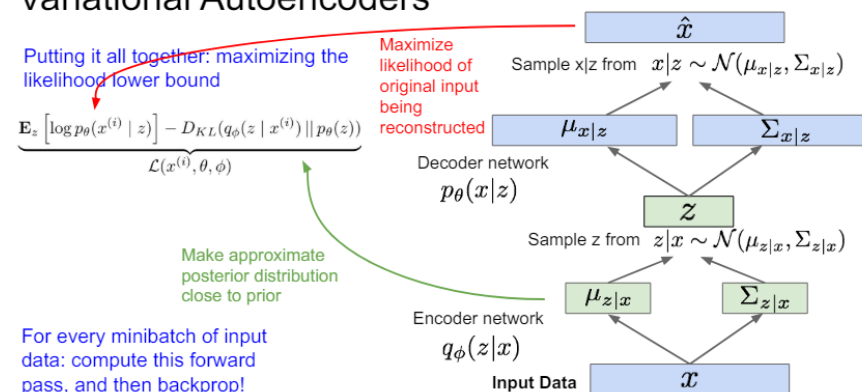


Autoenkodori predstavljaju nenadgledani pristup ka učenju reprezentacije podataka u prostoru niže dimenzionalnosti. Arhitektura autoenkodera se sastoji od enkodera i dekodera. Encoder na svom ulazu uzima podatke x i mapira ih u fičere z . Ovi fičeri imaju manju dimenzionalnost od

ulaznih podataka x (redukcija dimenzionalnosti). Drugi deo arhitekture je dekodera. Na ulaz dekodera se dovode fičeri z (podaci nakon redukcije dimenzionalnosti), a na njegovom izlazu se javljaju rekonstruisani originalni podaci na osnovu ovih fičera. Rekonstruisani podaci $\hat{x}$ su iste dimenzionalnosti kao i ulazni podaci x i cilj ovih mreža je da rekonstrukcija bude što bliža originalnim vrednostima. I enkoder i dekodera su neuronske mreže (danas se koriste fully connected i CNN mreže, pri čemu se kod CNN dekodera koristi up-convolucija). Autoenkoderi se treniraju tako da rekonstruisani podaci budu isti kao i ulazni podaci i u tu svrhu se koristi L2 loss. Ovakav način treniranja predstavlja nenadgledani pristup ka učenju korisnih fičera.

38. [DL09 Generative Models] Grafički predstaviti i opisati koja je specifičnost varijacionih autoenkodera (engl. Variational Autoencoders – VAE) u odnosu na obične autoenkodere. Koja im je osnovna namena i šta su prednosti i nedostaci njihovog korišćenja?

Variational Autoencoders



VAE predstavljaju mreže koje vrše generisanje novih slika (novih podataka). Fičeri z se mogu intuitivno shvatiti kao latentni faktori slike (npr. Ako je x slika lica, pojedinačne vrednosti z bi mogle da predstavljaju koliko je lice nasmevano, razmak između očiju itd.) VAE modeluju probabalističko generisanje podataka. Kod AE, enkoder mapira x na fičere z , a dekodera fičere z na rekonstrukciju originalnih podataka. Kod VAE, takođe imamo enkoder i dekodera, ali su oni probabalistički. Enkoder mreža $q_\phi(z|x)$ na izlazu daje srednju vrednost (mean) i dijagonalni kovarijansu za $z|x$, a na izlazu dekodera $p_\theta(x|z)$, a dekodera mreža na izlazu daje srednju vrednost (mean) i dijagonalnu kovarijansu za $x|z$ (NAPOMENA: $a|b$ znači a za dato b). Ovako definisane enkoder i dekodera mreže daju distribucije za x i z , respektivno. Na osnovu ovih distribucija se može izvršiti semplovanje i na taj način možemo generisati nove podatke. Generisanje novih podataka se vrši tako što se prvo obavi semplovanje z iz distribucije na osnovu izlaza enkodera, a zatim tako semplovani fičeri z propuštaju kroz dekodera i konačno se $\hat{x}$ semplove na osnovu distribucije sa izlaza dekodera.

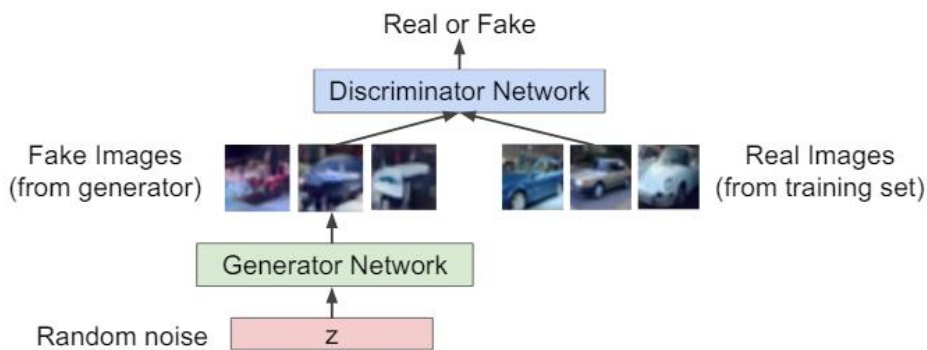
Prednosti:

- Predstavljaju principijalan pristup ka generativnim modelima
- Određivanje fičera $q(z|x)$, koji mogu biti korisni za razne druge zadatke (predstavljaju interpretabilne faktore varijacije)

Mane:

- Vršu maksimizaciju donje granice likelihood-a, što funkcioniše, ali nije najbolji mogući pristup
- Generisane slike su dosta mutnije i nižeg kvaliteta u odnosu na one dobijene korišćenjem novijih modela kao što su GAN-ovi

39. [DL09 Generative Models] Grafički predstaviti i opisati glavne elemente arhitekture generativnih protivničkih mreža (engl. Generative Adversarial Networks – GAN). Na koji način se obučavaju ove mreže?



GAN-ovi se sastoje iz dva dela: generator i diskriminator. Ova arhitektura se može shvatiti kao igra sa dva protivnika. Generator pokušava da zavarava diskriminatora tako što generiše realistične slike, a diskriminator pokušava da nauči razliku između pravih i lažnih slika. Treniraju se po principu min-max igre (gubitak jednog je dobitak drugog i obrnuto). Diskriminator se trenira tako da je njegov izlaz za prave podatke 1 (što označava prave podatke), a izlaz za podatke koje je generisao generator 0 (što je oznaka za lažne podatke). Generator se obučava tako da izlaz diskriminatora za generisane podatke bude 1 (cilj generatora je da zavarava diskriminator).

40. [DL10 Visualizing and Understanding] Za koje slojeve u konvolucionim neuronskim mrežama je moguće direktno vizualizovati težine? Objasniti na koji način se to postiže?

Težine je moguće direktno vizualizovati za konvolucione slojeve. Vizualizacija se postiže prikazivanjem filtera (kernela) kao slike, jer su filteri matrice koje se mogu tretirati kao pikseli slike. Za RGB slike, filteri se prikazuju kao 3-kanalne (u boji) slike.

U konvolucionoj mreži, ulazne težine za dati neuron imaju oblik [širina, visina, ulazni kanali]. Osim ako se ne radi o prvom konvolucionom sloju, ovo verovatno ne može biti lako vizualizovano jer je broj ulaznih kanala veliki (ako se skaliraju težine -> može, nego se samo filteri prvog sloja odnose na ulazne slike i težine prvog sloja, filteri u drugom sloju se odnose na izlaze prvog sloja, a ne ulaze i slike koje su nam od interesa itd.). Dakle, vizuelizacija se obično vrši samo ako je ovo prvi konvolucioni sloj! (Tehnički, za više slojeve može se koristiti redukcija dimenzionalnosti da se broj ulaznih kanala smanji na 3 dimenzije, ali baja priča za prvi).

41. [DL10_Visualizing_and_Understanding] Na koji način je moguće vizualizovati „prostor“ vektora obeležja (engl. feature vectors) koji se dobijaju na izlazu konvolucionog dela neuronske mreže i na osnovu kojih se vrši klasifikacija slika?

FC7 vektori karakteristika predstavljaju duboke reprezentacije podataka, često korišćene u složenim zadacima kao što su prepoznavanje objekata ili klasifikacija slika. Međutim, ovi vektori često imaju visok broj dimenzija, što ih čini teškim za analizu i vizualizaciju. FC7 vektori, na primer, imaju čak 4096 dimenzija.

Commented [8]: je li ovo sve?

Uzimo za primer AlexNet mrežu koja na izlazu, ispred klasifikatora, reprezentuje ulaznu sliku kao 4096-dimenzionalni vektor karakteristika na osnovu koga se vrši klasifikacija. Jedan od načina razumevanja i vizualizacije CNN jeste razumeti šta se dešava u poslednjem sloju. **Ideja je da se kroz tu mrežu propusti što više slika i da se prikupe vrednosti 4096-dim vektora karakteristika koji se dobije na izlazu.** Nakon toga, sve zabeležene vrednosti vektora možemo veoma lako vizualizovati primenom Nearest Neighbors metoda.

Rezultat ovog metoda prikazan je na slici. Kao što se vidi, slike su jako dobro podeljene, u neposrednoj blizini jesu ekvivalentni objekti iako se nalaze sa različitih strana slike. To je zbog toga što mreža sama određuje karakteristične tačke na osnovu kojih kasnije vrši poklapanje. Nasuprot ovome, da se ovaj metod primenio u prostoru piksela, u neposrednoj blizini bi se našle slike koje poseduju slične boje i konture iako se na njima ne nalaze isti objekti.

Osim ovog metoda, može se primeniti tehnika redukcije dimenzionalnosti kao što je PCA ili t-SNE.

Da bismo bolje razumeli ove visoko-dimenzionalne podatke, potrebno je smanjiti njihov broj dimenzija na nešto što možemo vizualizovati i analizirati. Vizualizacija prostora vektora obeležja se može postići korišćenjem tehnika redukcije dimenzionalnosti kao što su t-SNE ili PCA. Ove metode projektuju visoko-dimenzionalne vektore obeležja u 2D ili 3D prostor, omogućavajući vizualizaciju i analizu distribucije vektora. Jedan jednostavan i efikasan metod je Analiza glavnih komponenti (PCA). PCA transformiše podatke tako da sačuva što je moguće više varijance u samo nekoliko dimenzija. Ovaj algoritam može smanjiti 4096-dimenzionalne FC7 vektore na samo 2 dimenzije, omogućavajući nam da ih lako prikažemo na grafiku.

Međutim, ako želimo još precizniju vizualizaciju koja može da otkrije kompleksnije strukture i odnose između podataka, možemo koristiti t-SNE (t-Distributed Stochastic Neighbor Embedding). t-SNE je složeniji algoritam koji je posebno koristan za vizualizaciju visokodimenzionalnih podataka u 2 ili 3 dimenzije, jer bolje očuva lokalne odnose između tačaka.

42. [DL10_Visualizing_and_Understanding] Na koji način je moguće vizuelizovati koji delovi slika maksimalno aktiviraju pojedine mape obeležja u konvolucionoj neuronskoj mreži?

Aktivacije su vrednosti koje neuronske mreže generišu u određenim slojevima kao odgovor na ulazne podatke. **Aktivacione mape** su vizualni prikazi tih vrednosti, koji pokazuju kako različiti delovi ulazne slike utiču na izlaz svakog neurona u sloju. Aktivacione mape pomažu da se bolje razume koje karakteristike slike mreža prepoznaje i kako ih koristi za klasifikaciju.

Postupak glasi ovako:

Odabere se određeni sloj u mreži i određeni kanal. Na primer, sloj 5 mreže AlexNet ima 128 kanala 13x13. Biramo na primer kanal 17. Propustimo mnogo slika kroz mrežu i pratimo vrednosti izabranog kanala. Nakon toga se vizualizuju **delovi slike koji odgovaraju maksimalnoj aktivaciji nekih neurona**. Sortiraju se slike po aktivaciji. U ovom primeru, za sloj 5, se vidi da se maksimalno aktiviraju krugovi i kružni oblici na slici, kao što su oči ili njuška kod životinja. Poređenja radi, da je izabran neki raniji kanal maksimalnu aktivaciju bi izazvao neki primitivniji oblik sličan liniji, dok bi u daljim slojevima aktivaciju izazvao neki složeniji oblik kao što je lice.

Mape salijencije su tehnike koje se koriste za identifikaciju delova slike koji su najvažniji za odluku koju donosi neuronska mreža. One pokazuju kako bi promena svakog piksela uticala na izlaz modela, omogućavajući razumevanje koje regije slike najviše doprinose konačnoj klasifikaciji.

Postupak

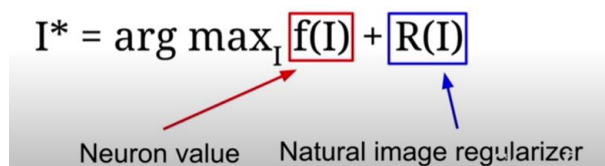
Propustanje slike: Propustimo sliku kroz mrežu.

Izračunavanje gradijenta: Izračunavamo gradijent izlaza mreže u odnosu na ulazne piksele. Ovo nam daje informaciju o tome kako mala promena u svakom pikselu utiče na izlaz.

Kreiranje mape salijencije: Gradijenti se koriste za kreiranje mape koja vizualno prikazuje koje regije slike su najvažnije za donošenje odluke.

Vizualizacija mape: Mapa salijencije se može vizualizovati kao toplotna mapa preko originalne slike, ističući najvažnije delove slike.

43. [DL10 Visualizing and Understanding] Na koji način je moguće vizuelizovati funkciju jednog neurona (pozicije u mapi obeležja) korišćenjem gradijentnog uspona (engl. gradient ascent)? Opisati korake po kojima se sprovodi ovaj postupak.

$$I^* = \arg \max_I \boxed{f(I)} + \boxed{R(I)}$$


Neuron value Natural image regularizer

Gradijentni uspon generiše sintetičku sliku koja maksimalno aktivira neuron.

1. Inicijalizacija slike na nule.

Ponavljati:

2. Proslediti sliku i izračunati vrednosti.

3. Backprop. radi dobijanja gradijenata prema vrednostima piksela (cilj je naći deo slike na koji neuron najviše reaguje).
4. Ažurirati sliku za nijansu.

Postoje načini da se poboljšaju vizualizacije. Jedan od primera jeste predstavljen algoritam regularizacije koji čini da generisane slike deluju dosta prirodnije. Bez regularizatora bi slika svakako maksimalno aktivirala neuron, ali ne bi ličila ni na šta i ne bi imala smisla čoveku, tako da je regularizacija dodata zbog vizualizacije.

$$\arg \max_I S_c(I) - \lambda \|I\|_2^2$$

Simple regularizer: Penalize L2 norm of generated image

Iznad je dat primer jednostavnog regularizatora - jednosavan, ništa spec, ali je jedan od najranijih koji se koristio, poboljšava sliku.

Commented [9]: treba li 43.minut

44. [DL10_Visualizing_and_Understanding] Na koji način je moguće prevariti konvolucioni neuronsku mrežu da pogrešno klasifikuje sliku (engl. adversarial examples)? Opisati korake po kojima se sprovodi ovaj postupak.

Kul način da se iskoristi priča sa gradijentima.

1. Počnite od proizvoljne slike.
2. Izaberite proizvoljnu klasu.
3. Modifikujte sliku kako bi se maksimalizovala klasa.

Ovaj korak uključuje izračunavanje gradijenta funkcije gubitka u odnosu na piksele slike, slično kao što to radimo za težine tokom treniranja mreže. Ali ovde, umesto ažuriranja težina, ažuriramo same piksele slike.

4. Ponavljajte ove korake dok se mreža ne prevari.

Na kraju, mreža će se prevariti ali nikakvih značajnih modifikacija na slici neće biti, odnosno ukoliko se krene od slike slona i teži na koali, slon neće postati crno beo i dobiti uši koale, naprotiv, **različiti će biti samo šumovi.**

Adversarial examples - (suprotstavljeni primeri) su specijalizovani ulazi kreirani sa svrhom zbunjivanja neuronske mreže, što rezultira pogrešnom klasifikacijom datog ulaza. Ovi ozloglašeni ulazi se ne mogu razlikovati za ljudsko oko, ali uzrokuju da mreža ne uspe da identifikuje sadržaj slike. Postoji nekoliko tipova ovakvih napada, međutim, ovde je fokus na napadu metode brzog gradijenta, što je napad bele kutije čiji je cilj da obezbedi pogrešnu klasifikaciju. Napad bele kutije je mesto gde napadač ima potpuni pristup modelu koji je napadnut. Jedan od najpoznatijih primera suprotstavljenih slika prikazane u nastavku preuzet je iz gore pomenutog rada.

Primer: Počevši od slike pande, napadač dodaje male perturbacije (izobličenja) originalnoj slici, što rezultira modelom koji ovu sliku označava kao gibona, sa visokim poverenjem. Proces dodavanja ovih perturbacija je objašnjen u nastavku.

45. [DL10_Visualizing_and_Understanding] Šta predstavlja i na koji način radi DeepDream algoritam za kreiranje „halucinacija“ na osnovu slika?

DeepDream algoritam je tehnika koja koristi konvolucione neuronske mreže (CNN) za generisanje vizuelnih efekata poznatih kao "halucinacije". Ovaj algoritam je razvijen od strane Google DeepMind tima.

DeepDream koristi već naučene CNN modele, kao što je VGG16, koji su obučeni na velikim skupovima slika. Ideja algoritma je da se **optimizuje ulazna slika** kako bi se **maksimalizovale aktivacije određenih neurona u mreži**.

Kako radi DeepDream algoritam?

Odabir mreže: Prvo se bira duboka neuronska mreža, na primer VGG, koja je predviđena za klasifikaciju slika. Ova mreža je pretrenirana na velikom skupu slika.

Odabir slojeva i aktivacija: Zatim se biraju slojevi unutar mreže koji su bliži ulazu i sadrže informacije o oblicima i teksturama na slici. Odabir slojeva zavisi od željenih efekata i stila koje želite da postignete. Primera radi, ukoliko se odaberu početni slojevi, biće prikazani jednostavniji oblici, dok će u slučaju kasnijih slojeva biti prikazani složeniji objekti.

Propagacija unapred: Ulazna slika se prosleđuje kroz mrežu i računaju se aktivacije neurona u odabranim slojevima.

Optimizacija: Nakon što se izračunaju aktivacije, koristi se metod optimizacije gradijentima kako bi se izračunao gradijent gubitka u odnosu na ulaznu sliku. **Ovaj gradijent predstavlja informaciju o tome kako treba ažurirati piksele ulazne slike** kako bi se pojačala aktivacija neurona u odabranim slojevima.

Iterativna optimizacija: Proces obrnutog propagiranja i ažuriranja piksela ulazne slike se ponavlja kroz više iteracija. Svaka iteracija dodaje male izmene na sliku u smeru gradijenta gubitka. To dovodi do stvaranja izraženih oblika i tekstura na slici, što rezultuje halucinacijama ili psihodeličnim efektima.

Rezultat: Kada se završi iterativna optimizacija, dobija se krajnja slika koja sadrži psihodelične oblike i teksture koje su bile aktivirane u odabranim slojevima mreže.

46. [DL10_Visualizing_and_Understanding] Na koji način funkcioniše algoritam za transfer stila sa jedne slike na drugu (engl. neural style transfer)? Na koji način se predstavlja stil, a na koji sadržaj slike?

Prilikom transfera stila, postoje dve slike koje učestvuju: slika sadržaja i slika stila. Slika sadržaja predstavlja sliku na koju će se preneti stil, dok slika stila predstavlja sliku čiji stil želimo da prenesemo.

Koriste se predobučeni modeli CNN-a, kao što su VGG ili InceptionNet, koji su obučeni na velikim skupovima slika.

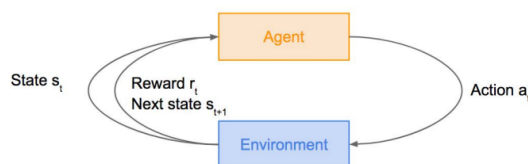
Stil se predstavlja kao statističke karakteristike izabranog stila slike, na primer, tekstura, boja i šabloni. One se izdvajaju korišćenjem konvolucionih neuronskih mreža, prolazeći kroz mrežu i izdvajajući aktivacione mape sa različitih slojeva. Ove **aktivacione mape sadrže informacije o stilu**.

Sadržaj slike se predstavlja informacijama o oblicima i strukturi slike. Ovo se takođe obavlja korišćenjem konvolucionih neuronskih mreža, ali se **izdvajaju aktivacione mape sa dubljih slojeva mreže** koje kodiraju informacije o sadržaju.

Algoritam za transfer stila kombinuje stil aktivacionih mapa i sadržaj aktivacionih mapa da bi generisao novu sliku koja ima stil sličan stilu referentne slike i sadržaj sličan sadržaju referentne slike. Ovaj proces se obično izvodi korišćenjem gradijentnog spusta ili drugih optimizacionih metoda da bi se minimizovao gubitak između generisane slike i referentnih stilskih i sadržajnih aktivacionih mapa.

47. [DL11_Reinforcement_Learning] Grafički predstaviti i opisati osnovne elemente koji se koriste kod pojačanog učenja (engl. reinforcement learning). Na primeru inverznog klatna pobrojati konkretne elemente po kategorijama.

Pojačano učenje - problemi koji uključuju agenta koji interaguje sa okruženjem, koje pruža numeričke signale nagrade. Cilj: Naučiti kako preduzimati akcije kako bi se maksimizirala nagrada.



Problem klatna

Cilj: Balansirati klatno na vrhu pokretnih kolica

Stanje: ugao, ugaona brzina, položaj, horizontalna brzina

Akcija: horizontalna sila primenjena na kolica

Nagrada: 1 pri svakom vremenskom koraku ako je klatno uspravno

48. [DL11_Reinforcement_Learning] Grafički ilustrovati i opisati arhitekturu Q-mreže koja je iskorišćena za obučavanje agenata da igraju klasične Atari igre. Na koji način se obučavaju Q-mreža (engl. Deep Q-Learning)?

Arhitektura Q-mreže:

Ulazni sloj - Prima stanje igre kao sliku (niz piksela).

Konvolucionni slojevi - Ekstrahuju fičere iz slike.

Fully Connected slojevi - Kombinuju fičere i procenjuju Q-vrednosti za svaku akciju.

Obučavanje Q-mreže:

Korak 1: Inicijalizacija Q-mreže sa slučajnim težinama.

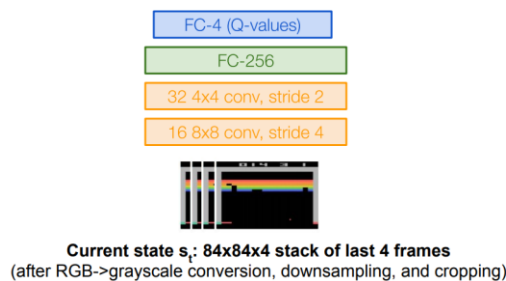
Korak 2: Igrajući igru, prikupljanje iskustava (stanje, akcija, nagrada, sledeće stanje).

Korak 3: Ažuriranje Q-vrednosti koristeći Bellmanovu jednačinu: $Q(s,a) = r + \gamma \max_a Q(s',a)$

Korak 4: Koristeći metode poput mini-batch gradijentnog spusta za optimizaciju mreže.

Q-network Architecture

$Q(s, a; \theta)$:
neural network
with weights θ



Experience Replay (Ponovno Igranje Iskustava)

Experience replay je tehnika korišćena u algoritmima za pojačano učenje, poput Deep Q-Learning-a, koja omogućava efikasnije i stabilnije učenje agenata. Ova tehnika uključuje čuvanje iskustava agenta u bazi podataka nazvanoj replay buffer (bafer za ponovno igranje) i njihovo ponovno korišćenje tokom obuke

49. Koje su ključne karakteristike transformer arhitekture za obradu prirodnog jezika?

BRT GDE IMA OVO DA SE NAĐE

Self-Attention Mehanizam - Omogućava modelu da proceni važnost svakog dela ulazne sekvence u odnosu na druge delove sekvence. To omogućava paralelizaciju obrade, za razliku od rekurentnih mreža.

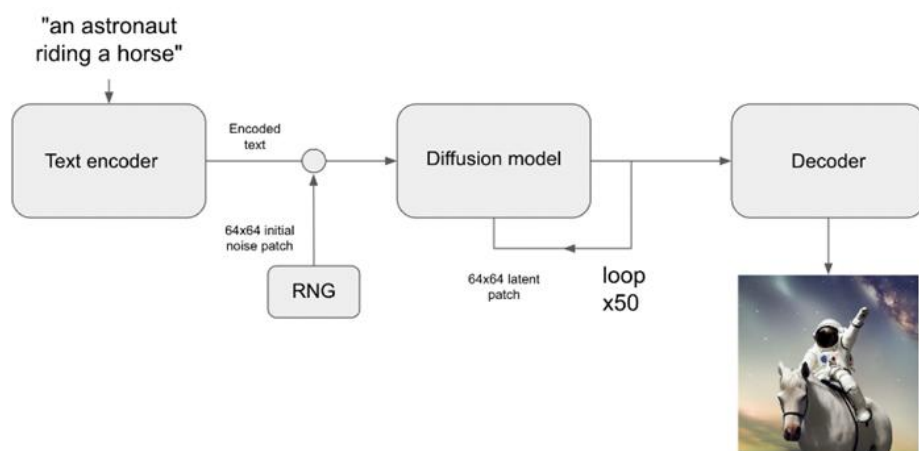
Positional Encodings - Dodaju informacije o redosledu elemenata u sekvenci jer transformeri ne prate prirodno redosled reči. Ove enkodiranja se dodaju ulaznim vektorima kako bi model mogao da razlikuje različite pozicije u sekvenci.

Multi-head Self-Attention - Omogućava modelu da istovremeno fokusira na različite delove sekvence iz različitih perspektiva. Svaka "glava" u self-attention mehanizmu uči različite reprezentacije sekvence.

Feed-Forward Neural Networks - Linearne transformacije i nelinearne aktivacije koje se primenjuju na izlaz iz self-attention slojeva za dodatnu obradu informacija.

Enkoder-Dekoder Struktura - Za zadatke kao što su prevođenje, transformeri koriste enkoder-dekoder strukturu. Enkoder obrađuje ulaznu sekvencu i stvara kontekstualne reprezentacije, dok dekoder koristi te reprezentacije za generisanje izlazne sekvence.

50. Grafički predstaviti i opisati princip rada stable diffusion algoritma za generisanje slika na osnovu tekstualnog opisa.



(RNG - random noise generator), source: [High-performance image generation using Stable Diffusion in KerasCV | TensorFlow Core](#)

Stable Diffusion je algoritam za generisanje slika koji koristi difuzione modele. Evo grafičkog prikaza i opisa principa rada:

Ulazni Tekst - Algoritam prima tekstualni opis kao ulaz (npr. "mačka koja nosi šešir").

Enkodiranje Teksta - Tekst se enkodira u latentni prostor koristeći tekstualni enkoder (obično transformer).

Generisanje Slike - Početni latentni prostor se inicijalizuje sa slučajnim šumom. Kroz niz iteracija, model uklanja šum i rafinira latentni prostor prema cilju generisanja slike koja odgovara tekstualnom opisu.

Dekodiranje Latentnog Prostora - Latentni prostor se dekodira natrag u sliku visoke rezolucije pomoću difuzionog modela.

Stable Diffusion koristi kombinaciju dubokih generativnih modela i tehnika difuzije kako bi stvorio slike koje odgovaraju zadatom tekstualnom opisu. Proces uključuje iterativno uklanjanje šuma iz latentnog prostora, što omogućava generisanje visokokvalitetnih slika.

Iz mog seminarskog, ukratko o stable diffusion:

Princip rada ovog modela se zasniva na neuronskim mrežama čija je uloga da izvrše denoising i povećanje rezolucije slike (super-resolution). Modeli koji obavljaju ovaj zadatak ne mogu izvući informacije koje ne postoje iz slike niske rezolucije, već ovo rade tako što detalje koji nisu prisutni u originalnoj slici „haluciniraju“ na osnovu trening skupa podataka. Ako bismo ovakav model primenili nad čistim šumom, model bi „halucinirao“ ceo sadržaj slike i time generisao novu sliku, ponavljanjem postupka denoising-a. Modeli latentne difuzije u ovakav sistem uvode kontrolu putem tekstualnog ulaza. Komponente ovog sistema su prikazane na slici X. Enkoder teksta pretvara tekstualni ulaz u vektor unutar odgovarajućeg latentnog vektorskog prostora. Zatim, na ovaj vektor se konkatenuira nasumično generisano parče šuma, nad kojim se više puta primenjuje denoising postupak koji obavlja model difuzije. Na kraju, dekoader renderuje latentnu sliku na izlazu modela difuzije u visokoj rezoluciji.